

UNIVERSIDAD TÉCNICA DE MACHALA

FACULTAD DE INGENIERÍA CIVIL

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

INTELIGENCIA ARTIFICIAL

Reproducing Images Pygad

CURSO

Decimo Semestre Paralelo “A”

ESTUDIANTE

Mario Sebastián Ávila León

DOCENTE TUTOR

Ing. Sist. Rivas Asanza Wilmer Braulio, Phd.

PERIODO LECTIVO

2026 - 2

Índice

1. Introducción 1

2. Objetivos 3

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Marco teórico 5

3.1. Algoritmos genéticos 5

3.2. Algoritmos genéticos aplicados al procesamiento de imágenes 7

3.3. PyGAD 8

3.4. GARI (Genetic Algorithm for Reproducing Images) 9

4. Materiales y herramientas 11

5. Metodología experimental 13

6. Configuración del algoritmo genético 15

7. Descripción del código 19

7.1. Instalación e importación de dependencias 19

7.2. Carga y normalización de la imagen 19

7.3. Conversión de la imagen objetivo a cromosoma 19

7.4. Función de aptitud 20

7.5. Redefinición local (redundante) de las funciones de GARI 20

7.6. Configuración y ejecución del algoritmo genético 21

7.7. Visualización de resultados 22

8. Características de la imagen original 23

8.1. Inconsistencia detectada sobre la procedencia de la imagen 23

8.2. Características técnicas de la imagen utilizada 24

8.3. Relevancia de una imagen de 100 × 100 píxeles en este experimento 25

8.4. Cita del repositorio Fruits-360 y de su licencia 26

9. Experimentos realizados 26

10. Tiempos de ejecución 28
10.1. Tiempo promedio por generación 29
10.2. Relación entre el número de generaciones y el tiempo total 30
11. Resultados 31
11.1. Gráfica 1: número de generaciones frente al tiempo de ejecución 31
11.2. Gráfica 2: tiempo promedio por generación 32
11.3. Tabla comparativa entre generaciones 33
12. Comparación de imágenes 37
12.1. Experimento de 1 generación 37
12.2. Experimento de 100 generaciones 38
12.3. Experimento de 1.000 generaciones 39
12.4. Experimento de 10.000 generaciones 40
12.5. Experimento de 25.000 generaciones 41
12.6. Experimento de 50.000 generaciones 42
13. Análisis visual 44
13.1. Experimentos de 1 y 100 generaciones 44
13.2. Experimento de 1.000 generaciones 44
13.3. Experimento de 10.000 generaciones 45
13.4. Experimento de 25.000 generaciones 45
13.5. Experimento de 50.000 generaciones 46
13.6. Síntesis comparativa de la evolución visual 46
14. Análisis de eficiencia 47
14.1. Número de genes y su influencia en el costo computacional 47
14.2. Influencia del tamaño de la población 47
14.3. Influencia del número de generaciones 48
14.4. Por qué 50.000 generaciones requieren considerablemente más tiempo 48
14.5. Posibles cuellos de botella del código 49

14.6. Oportunidades de optimización 49
15. Análisis de convergencia 50
16. Discusión 51
17. Conclusiones 55
18. Recomendaciones 57
19. Referencias bibliográficas 59
20. Anexos 60
20.1. Anexo A. Función de aptitud completa (fitness_fun) 60
20.2. Anexo B. Configuración completa de pygad.GA 60
20.3. Anexo C. Funciones principales de gari.py 60
20.4. Anexo D. Tabla completa de tiempos de ejecución (archivo tiempos.txt) 61
20.5. Anexo E. Relación de archivos de imagen entregados 62

1. Introducción

El presente informe documenta, analiza y discute de manera técnica y académica el desarrollo de un proyecto de reproducción de imágenes mediante un algoritmo genético (AG), implementado en el lenguaje de programación Python con apoyo de la biblioteca PyGAD y del módulo auxiliar GARI (Genetic Algorithm for Reproducing Images). El proyecto analizado corresponde a un cuaderno de trabajo de tipo Jupyter/Google Colab denominado pygad_imagen.ipynb, acompañado del módulo gari.py, de un archivo de texto con los tiempos de ejecución aproximados de seis experimentos independientes y de las imágenes generadas durante dichas ejecuciones.

Los algoritmos genéticos constituyen una familia de métodos de optimización y búsqueda heurística inspirados en los mecanismos de la selección natural y la genética mendeliana. En el campo del procesamiento digital de imágenes, estos algoritmos pueden emplearse para resolver, de manera experimental, el problema de reconstruir progresivamente una imagen objetivo a partir de una población de soluciones candidatas generadas aleatoriamente, donde cada candidato evoluciona generación tras generación mediante los operadores de selección, cruce y mutación, hasta aproximarse, en mayor o menor medida, a la imagen original.

El objeto de estudio de este informe es, por tanto, doble: por un lado, se examina el fundamento conceptual y la implementación técnica del algoritmo genético empleado, parámetros de PyGAD, representación cromosómica de la imagen mediante GARI y función de aptitud utilizada, por otro lado, se analizan los resultados empíricos obtenidos en seis experimentos ejecutados con un número creciente de generaciones (1; 100; 1.000; 10.000; 25.000 y 50.000), evaluando tanto la evolución visual de la imagen reconstruida como el comportamiento del tiempo de cómputo requerido en cada caso.

Es importante señalar, desde esta introducción, que el análisis se ha construido exclusivamente a partir de la evidencia disponible en los archivos suministrados, el propio código fuente, el archivo de tiempos y las imágenes resultantes, evitando en todo momento la incorporación de parámetros, resultados o métricas no verificables. En los apartados correspondientes se documentan explícitamente las inconsistencias detectadas entre lo indicado en el enunciado del trabajo y lo observado directamente en los archivos del proyecto (por ejemplo, en lo relativo al nombre y procedencia real de

la imagen original), de manera que el lector pueda distinguir con claridad entre el dato verificado y la interpretación razonada del mismo.

El informe se organiza en veinte secciones numeradas, precedidas por esta introducción y por los objetivos del trabajo, y seguidas por el marco teórico, la descripción de materiales y metodología, la configuración detallada del algoritmo genético, la descripción del código fuente, las características de la imagen original, la descripción de los experimentos realizados, el análisis de los tiempos de ejecución, la presentación de resultados (tablas y gráficas), la comparación de imágenes por experimento, el análisis visual, el análisis de eficiencia computacional, el análisis de convergencia, la discusión académica, las conclusiones, las recomendaciones para trabajos futuros, las referencias bibliográficas en formato IEEE y, finalmente, los anexos con fragmentos relevantes del código.

Link del repositorio: **https://github.com/MarioAvilaSL/Reproducing-Images-Pygad**

2. Objetivos

2.1. Objetivo general

Analizar de manera técnica, documentada y reproducible el proyecto de reproducción de una imagen mediante un algoritmo genético implementado con la biblioteca PyGAD y el módulo GARI, evaluando su fundamento teórico, su configuración experimental, su desempeño computacional y la evolución visual de los resultados obtenidos a lo largo de distintos números de generaciones.

2.2. Objetivos específicos

- Describir el fundamento teórico de los algoritmos genéticos y su aplicación específica al procesamiento y la reconstrucción de imágenes digitales.
- Examinar en detalle el código fuente del proyecto (pygad_imagen.ipynb y gari.py), identificando las librerías empleadas, la representación de la imagen y del cromosoma, la función de aptitud y los parámetros de configuración de PyGAD realmente utilizados.
- Documentar de forma exhaustiva las características de la imagen original empleada en el experimento.
- Describir la metodología experimental seguida, de manera que el procedimiento pueda ser reproducido por un tercero.
- Analizar los tiempos de ejecución reportados para los seis experimentos (1; 100; 1.000; 10.000; 25.000 y 50.000 generaciones), calculando métricas derivadas como el tiempo promedio por generación y el incremento porcentual entre experimentos.
- Presentar y comparar visualmente las imágenes obtenidas en cada experimento, relacionando los cambios observados con el funcionamiento interno del algoritmo genético.
- Analizar la eficiencia computacional del algoritmo en función del número de genes, el tamaño de la población y el número de generaciones.
- Discutir, desde una perspectiva académica y objetiva, si el incremento del número de generaciones y del tiempo de cómputo se traduce en mejoras visuales proporcionales.

- Formular conclusiones sustentadas exclusivamente en la evidencia disponible y recomendaciones técnicas para futuros experimentos.

3. Marco teórico

3.1. Algoritmos genéticos

Un algoritmo genético (AG) es un método de búsqueda y optimización estocástica perteneciente a la familia de los algoritmos evolutivos, inspirado en los principios de la evolución biológica descritos por Charles Darwin y formalizados computacionalmente por John Holland en la década de 1970 [1]. Su funcionamiento se basa en mantener una población de soluciones candidatas a un problema, los individuos, que evoluciona iterativamente mediante operadores inspirados en la genética natural: selección, cruce (o recombinación) y mutación, con el fin de mejorar progresivamente la calidad de las soluciones según una medida cuantitativa denominada función de aptitud (fitness function), tal como se sistematiza en los trabajos clásicos de Goldberg [2] y Mitchell [3].

La inspiración biológica del algoritmo se sustenta en la analogía entre los conceptos de la genética de poblaciones y los elementos computacionales del AG: cada solución candidata se asimila a un individuo de una población; el conjunto de individuos evaluados en una misma iteración constituye la población; la codificación interna de cada individuo, habitualmente un vector de valores numéricos o binarios, se denomina cromosoma, y cada componente individual de dicho vector recibe el nombre de gen. De manera análoga a como en la naturaleza los individuos mejor adaptados a su entorno tienen mayor probabilidad de reproducirse y transmitir su información genética, en el AG los individuos con mayor aptitud tienen mayor probabidad de ser seleccionados como padres para generar la siguiente generación de soluciones.

La función de aptitud (fitness function) es el componente que cuantifica qué tan buena es una solución candidata respecto del objetivo que se desea optimizar. Es, en la práctica, la pieza que traduce el problema real a un valor numérico comparable entre individuos, permitiendo ordenarlos y seleccionar a los más aptos. En el caso de problemas de maximización, a mayor valor de aptitud, mejor es la solución; cuando el problema es naturalmente de minimización, como ocurre al comparar una imagen candidata contra una imagen objetivo mediante una medida de error, es habitual transformar la métrica de error en una métrica de aptitud creciente, tal como se explica en la sección 3.3 y se detalla en la sección 7 de este informe.

El operador de selección determina qué individuos de la población actual participarán como padres en la generación de la descendencia. Existen distintos métodos de selección, selección por ruleta (roulette wheel selection), selección por rango (rank selection), selección por torneo (tournament selection) y selección por estado estacionario (steady-state selection), entre otros, cada uno con implicaciones distintas sobre la presión selectiva y la diversidad genética de la población a lo largo de las generaciones.

El operador de cruce (crossover) combina la información genética de dos (o más) individuos padres para producir uno o varios individuos descendientes, con la expectativa de que la descendencia herede y combine las características favorables de ambos progenitores. Entre las variantes más comunes se encuentran el cruce de un punto (single point), el cruce de dos puntos (two points), el cruce uniforme (uniform) y el cruce disperso (scattered).

El operador de mutación introduce variaciones aleatorias en uno o varios genes de un individuo, con una probabilidad generalmente baja. Su función principal es mantener la diversidad genética de la población y evitar la convergencia prematura hacia óptimos locales, permitiendo la exploración de regiones del espacio de búsqueda que no serían alcanzables únicamente mediante la recombinación de los individuos existentes.

Una generación corresponde a cada ciclo completo de evaluación, selección, cruce y mutación que produce una nueva población a partir de la anterior. El número de generaciones ejecutadas es, junto con el tamaño de la población, uno de los principales factores que determinan tanto la calidad final de la solución obtenida como el costo computacional total del algoritmo.

Finalmente, se entiende por convergencia el fenómeno por el cual la población de un AG tiende, a lo largo de las generaciones, a concentrarse en torno a soluciones de alta aptitud, reduciéndose la diversidad genética y estabilizándose la mejora observada en la función de aptitud. La convergencia puede ser deseable cuando se aproxima al óptimo global del problema, pero también puede resultar prematura si la población pierde diversidad antes de explorar adecuadamente el espacio de soluciones, quedando atrapada en un óptimo local. En la sección 15 de este informe se discute, con las debidas cautelas metodológicas, si los datos disponibles permiten identificar indicios de convergencia en el experimento analizado.

3.2. Algoritmos genéticos aplicados al procesamiento de imágenes

La aplicación de algoritmos genéticos a la reproducción de imágenes parte de un principio conceptual sencillo: una imagen digital puede representarse como una matriz numérica de dimensiones alto × ancho × canales de color, en la que cada elemento corresponde a la intensidad de un canal de color (por ejemplo, rojo, verde y azul en el modelo RGB) de un píxel determinado. Dado que un algoritmo genético opera sobre cromosomas representados habitualmente como vectores unidimensionales de valores numéricos, resulta natural establecer una correspondencia directa entre los píxeles (y sus canales de color) de la imagen y los genes del cromosoma.

El proceso de conversión de una imagen a un cromosoma requiere, en primer lugar, la conversión de la imagen a una matriz numérica, operación realizada de forma estándar por las bibliotecas de lectura de imágenes, como imageio, que devuelven un arreglo de NumPy. En segundo lugar, es habitual normalizar los valores de los píxeles, típicamente representados en el rango entero [0, 255] para imágenes de 8 bits por canal, a un rango continuo [0, 1], dividiendo cada valor entre 255. Esta normalización favorece la estabilidad numérica del algoritmo y permite acotar de manera consistente los rangos de inicialización y de mutación de los genes.

En tercer lugar, la matriz normalizada, de forma tridimensional (alto, ancho, canales), se convierte en un vector unidimensional mediante una operación de remodelado (reshape), que recorre de manera secuencial todos los elementos de la matriz y los dispone en una única fila. De este modo, cada gen del cromosoma corresponde exactamente a un canal de color de un píxel de la imagen: por ejemplo, en una imagen de 100 × 100 píxeles con 3 canales de color, el cromosoma resultante contendrá 30.000 genes (100 × 100 × 3).

La reconstrucción de la imagen a partir de un cromosoma consiste en la operación inversa: el vector unidimensional de genes se remodela nuevamente hacia la forma tridimensional original (alto, ancho, canales), reconstruyendo así una imagen que puede visualizarse y compararse directamente con la imagen objetivo. Esta operación exige que la longitud del vector coincida exactamente con el producto de las dimensiones de la forma destino, condición que, como se detalla en la sección 3.4, es verificada explícitamente en el código del módulo GARI.

La comparación entre la imagen objetivo y cada individuo generado por el algoritmo se realiza a nivel de cromosoma, es decir, comparando gen a gen (píxel a píxel y canal a canal) el vector del individuo candidato contra el vector de la imagen objetivo. La medida de discrepancia más simple, y la empleada en este proyecto, es la suma de las diferencias absolutas entre ambos vectores, la cual constituye una métrica de error: cuanto menor es esta suma, más parecido es el individuo a la imagen objetivo. Como se explica en la sección 3.3, esta métrica de error se transforma en una métrica de aptitud creciente para ser compatible con el criterio de maximización utilizado por PyGAD. La aplicación de algoritmos genéticos a tareas de mejora, segmentación y reconstrucción de imágenes ha sido documentada en diversos trabajos de revisión, entre ellos los de Paulinas y Ušinskas [4] y el de Mirjalili et al. [5], específicamente centrado en la reconstrucción de imágenes mediante algoritmos genéticos.

3.3. PyGAD

PyGAD es una biblioteca de código abierto para Python, orientada a la construcción, ejecución y personalización de algoritmos genéticos con fines de optimización, publicada por primera vez en abril de 2020 y documentada en detalle tanto en el artículo original de su autor [6] como en su documentación oficial en línea [7]. La biblioteca expone la clase pygad.GA, cuyo constructor admite un amplio conjunto de parámetros que permiten configurar prácticamente todos los aspectos del ciclo de vida del algoritmo: el tamaño de la población, el número de genes, los rangos de inicialización, el tipo y la probabilidad de selección de padres, el tipo de cruce, el tipo y el porcentaje de mutación, el número de generaciones y numerosos parámetros adicionales de control.

El uso típico de PyGAD, tal como se refleja en el cuaderno analizado, se estructura en tres pasos principales: (1) la definición de una función de aptitud personalizada, que PyGAD invoca automáticamente para evaluar cada individuo de la población; (2) la creación de una instancia de la clase pygad.GA, pasando como argumentos los parámetros de configuración del algoritmo; y (3) la invocación del método run() de dicha instancia, que ejecuta el ciclo completo de generaciones (selección, cruce, mutación y evaluación) hasta alcanzar el número de generaciones especificado o hasta que se cumpla un criterio de parada, si este ha sido definido.

En el proyecto analizado, la función de aptitud (fitness_fun) recibe tres parámetros, la instancia del algoritmo genético, la solución candidata (el cromosoma del individuo) y el índice de dicha solución dentro de la población, tal como exige la interfaz de PyGAD para las funciones de aptitud definidas por el usuario. Tras la ejecución de PyGAD, la instancia expone métodos como plot_fitness(), utilizado en el cuaderno para graficar la evolución de la aptitud del mejor individuo a lo largo de las generaciones, y best_solution(), utilizado para recuperar la mejor solución encontrada, su valor de aptitud y su índice dentro de la población final.

PyGAD contempla, además del módulo principal pygad.GA, submódulos especializados para el entrenamiento de redes neuronales mediante algoritmos genéticos (pygad.nn, pygad.gann, pygad.cnn, pygad.gacnn) y para la optimización de modelos construidos con Keras y PyTorch (pygad.kerasga, pygad.torchga); ninguno de estos submódulos adicionales es utilizado en el proyecto analizado, que emplea exclusivamente la clase base pygad.GA.

3.4. GARI (Genetic Algorithm for Reproducing Images)

GARI (acrónimo de Genetic Algorithm for Reproducing Images) es un pequeño proyecto complementario, también desarrollado por Ahmed Fawzy Gad, autor asimismo de PyGAD, cuyo propósito específico es facilitar la conversión bidireccional entre una imagen y su representación cromosómica, de modo que dicha representación pueda ser manipulada directamente por un algoritmo genético [9]. Este mismo caso de uso se documenta, con el mismo código y la misma imagen de referencia fruit.jpg, en el tutorial oficial de PyGAD dedicado a la reproducción de imágenes [8]. El archivo gari.py proporcionado en este proyecto contiene, exclusivamente, dos funciones: img2chromosome() y chromosome2img().

La función img2chromosome(img_arr) recibe como argumento un arreglo de NumPy que representa una imagen (potencialmente multidimensional, por ejemplo, con forma alto × ancho × canales) y devuelve un vector unidimensional, obtenido mediante numpy.reshape(), cuya longitud es igual al producto de todas las dimensiones del arreglo original (calculado internamente con functools.reduce y operator.mul). Esta función implementa, en la práctica, el paso de «conversión de la matriz a un vector unidimensional» descrito en la sección 3.2, y es la que se utiliza para obtener el cromosoma objetivo (target_chromosome) a partir de la imagen de referencia.

La función chromosome2img(vector, shape) realiza la operación inversa: recibe un vector unidimensional (por ejemplo, la mejor solución encontrada por PyGAD) y una forma de destino (shape), y devuelve un arreglo de NumPy remodelado según dicha forma. Antes de realizar el remodelado, la función valida explícitamente que la longitud del vector coincida con el producto de las dimensiones especificadas en shape; en caso contrario, lanza una excepción ValueError con un mensaje descriptivo del error. Esta verificación defensiva evita reconstrucciones erróneas o silenciosamente corruptas de la imagen cuando el número de genes del cromosoma no corresponde exactamente con las dimensiones esperadas.

En conjunto, ambas funciones constituyen el «puente» necesario entre el dominio de la imagen (matrices de píxeles) y el dominio del algoritmo genético (cromosomas representados como vectores), permitiendo que PyGAD manipule, evalúe y haga evolucionar los cromosomas sin necesidad de conocer su interpretación como imagen, mientras que GARI se encarga de traducir dicha representación en ambos sentidos cuando es necesario visualizar o evaluar el resultado.

Cabe señalar, como observación técnica derivada del análisis directo del código, que el cuaderno pygad_imagen.ipynb no solo importa el módulo gari (import gari) y utiliza sus funciones (gari.img2chromosome, gari.chromosome2img), sino que además redefine, en una celda posterior, un conjunto de funciones locales idénticas en su lógica (img2chromosome y chromosome2img) sin utilizarlas posteriormente en el flujo de ejecución. Esta duplicación no altera el resultado del experimento, dado que las funciones invocadas efectivamente son las importadas desde el módulo gari, pero constituye una redundancia de código que se documenta aquí por rigor descriptivo, sin que ello implique un error funcional del algoritmo.

4. Materiales y herramientas

El desarrollo del proyecto se apoya en un conjunto reducido de herramientas de software, todas ellas correspondientes al ecosistema científico de Python. A continuación, se listan, exclusivamente, las librerías efectivamente importadas en el cuaderno pygad_imagen.ipynb y en el módulo gari.py, según se constató mediante la revisión directa del código fuente.

Tabla 1. Librerías y herramientas utilizadas en el proyecto

Librería / módulo	Función dentro del proyecto	Observaciones
imageio (imageio.v2)	Lectura de archivos de imagen desde disco (función imread) y su conversión inicial a un arreglo de NumPy.	Se utiliza explícitamente la interfaz imageio.v2, indicada en el import como import imageio.v2 as imageio, lo que sugiere una adaptación a cambios de compatibilidad de versiones recientes de la librería.
numpy	Manipulación de arreglos multidimensionales: normalización de la imagen, cálculo de sumas y diferencias absolutas en la función de aptitud, y remodelado (reshape) de matrices y vectores.	Es la dependencia numérica central de todo el flujo de conversión imagen ↔ cromosoma.
gari (módulo local, gari.py)	Provee las funciones img2chromosome() y chromosome2img() para convertir la imagen en un vector (cromosoma) y viceversa.	Depende internamente de numpy, functools y operator.

matplotlib.pyplot	Visualización de la imagen reconstruida (imshow) y de la evolución de la aptitud a lo largo de las generaciones (a través del método plot_fitness() de PyGAD, que internamente utiliza matplotlib).	Es también la librería que genera los archivos de imagen entregados como resultado de cada ejecución.
pygad	Biblioteca principal del algoritmo genético: expone la clase pygad.GA empleada para instanciar, configurar y ejecutar el algoritmo evolutivo.	Instalada en el entorno de Colab mediante !pip install pygad, según se observa en la primera celda del cuaderno.
functools / operator (usadas dentro de gari.py)	Cálculo del producto de las dimensiones de un arreglo (functools.reduce junto con operator.mul), necesario para determinar la longitud del cromosoma y para validar la reconstrucción de la imagen.	Forman parte de la biblioteca estándar de Python; no requieren instalación adicional.

En cuanto al entorno de ejecución, la primera celda del cuaderno (!pip install pygad) indica que el proyecto fue desarrollado y ejecutado en un entorno de tipo Jupyter/Google Colab, en el cual las dependencias no forman parte del entorno base y deben instalarse explícitamente al inicio de cada sesión. No se dispone, en los archivos proporcionados, de información adicional sobre la versión exacta de Python, de PyGAD, ni sobre las características de hardware (CPU/GPU, memoria RAM) del entorno en el que se ejecutaron los experimentos; por consiguiente, este informe no

reporta dichos datos como si estuvieran verificados, y cualquier referencia a ellos en la discusión se presenta explícitamente como una limitación del análisis.

5. Metodología experimental

El procedimiento experimental documentado en el cuaderno pygad_imagen.ipynb sigue una secuencia lineal de diez etapas, descritas a continuación en el orden en que aparecen en el código, con el propósito de que el experimento pueda ser reproducido íntegramente por un tercero que disponga de los mismos archivos de entrada (fruit.jpg y gari.py).

1. Carga de la imagen original. Se lee el archivo fruit.jpg mediante imageio.imread(), obteniendo un arreglo de NumPy de forma (100, 100, 3) con valores enteros de 8 bits (0-255) en cada canal de color.

2. Normalización de la imagen. El arreglo se convierte a valores de punto flotante en el rango [0, 1] mediante la operación target_im = numpy.asarray(target_im / 255, dtype=float), homogeneizando la escala de los píxeles con el rango de inicialización y de mutación del algoritmo genético.

3. Conversión de la imagen a cromosoma. Se invoca gari.img2chromosome(target_im) para obtener target_chromosome, un vector unidimensional de 30.000 posiciones (100 × 100 × 3) que constituye la referencia contra la cual se evaluará cada individuo.

4. Definición de la función de aptitud. Se define fitness_fun(ga_instance, solution, solution_idx), que calcula la suma de las diferencias absolutas entre el cromosoma objetivo y el cromosoma de la solución evaluada, y transforma dicho error en un valor de aptitud creciente (véase la sección 7).

5. Inicialización de la población. Al crear la instancia de pygad.GA, no se proporciona una población inicial explícita (initial_population), por lo que PyGAD genera automáticamente sol_per_pop = 20 individuos, cada uno de 30.000 genes con valores aleatorios uniformemente distribuidos entre init_range_low = 0.0 e init_range_high = 1.0.

6. Evaluación de individuos. Cada individuo de la población es evaluado mediante fitness_fun, obteniendo su valor de aptitud, lo que permite ordenar la población de mejor a peor solución en cada generación.

7. Aplicación de los operadores genéticos. En cada generación, PyGAD selecciona num_parents_mating = 10 padres (según el método de selección por defecto, selección por estado estacionario), aplica el operador de cruce configurado por defecto (single_point) para producir la descendencia, y aplica mutación aleatoria por reemplazo sobre un porcentaje muy reducido de genes (mutation_percent_genes = 0.01 %, equivalente a aproximadamente 3 de los 30.000 genes por individuo).

8. Evolución durante las generaciones. El ciclo de evaluación-selección-cruce-mutación se repite de manera iterativa durante el número de generaciones configurado (num_generations), valor que, según la evidencia de los archivos de tiempos e imágenes entregados, fue modificado sucesivamente entre ejecuciones independientes para producir los seis experimentos de 1; 100; 1.000; 10.000; 25.000 y 50.000 generaciones analizados en este informe.

9. Reconstrucción de imágenes. Al finalizar cada ejecución, se recupera la mejor solución mediante ga_instance.best_solution() y se reconstruye la imagen correspondiente aplicando gari.chromosome2img(solution, target_im.shape), obteniendo un arreglo con la misma forma (100, 100, 3) que la imagen original.

10. Registro de resultados y comparación visual/temporal. La imagen reconstruida se visualiza mediante matplotlib.pyplot.imshow() y se exporta como archivo de imagen; adicionalmente, se invoca ga_instance.plot_fitness() para graficar la evolución de la aptitud del mejor individuo a lo largo de las generaciones. El tiempo total de cada ejecución fue registrado manualmente por el autor del proyecto en el archivo tiempos.txt, dado que el cuaderno no incluye instrucciones de medición automática del tiempo (por ejemplo, mediante el módulo time).

Para reproducir el experimento completo, un tercero debería: (a) disponer del archivo de imagen original fruit.jpg y del módulo gari.py en el mismo directorio de trabajo; (b) instalar las dependencias imageio, numpy, matplotlib y pygad; (c) ejecutar secuencialmente las celdas del cuaderno; y (d) modificar el parámetro num_generations de la instancia de pygad.GA antes de cada ejecución, si se desea reproducir cada uno

de los seis experimentos analizados en este informe, registrando manualmente el tiempo de cada ejecución, tal como parece haberse realizado en el proyecto original.

6. Configuración del algoritmo genético

La tabla siguiente resume los parámetros de configuración de la clase pygad.GA tal como se encuentran, de manera explícita y literal, en el código fuente del cuaderno pygad_imagen.ipynb (celda de creación de ga_instance). Únicamente se documentan aquí los valores efectivamente presentes en el código; cuando un parámetro relevante de PyGAD no fue especificado explícitamente por el autor del proyecto, se indica su valor por defecto según la documentación oficial de la biblioteca [7], señalándolo expresamente como tal.

Tabla 2. Parámetros de configuración de pygad.GA especificados explícitamente en el código

Parámetro	Valor	Función dentro del algoritmo	Impacto esperado
num_generations	50.000	Número total de generaciones (ciclos de evolución) que ejecutará el algoritmo.	Determina de forma directa el tiempo total de cómputo y, en principio, el grado de refinamiento alcanzable en la reconstrucción de la imagen.
num_parents_mating	10	Número de individuos seleccionados como padres en cada generación para producir la descendencia mediante cruce.	Con una población de 20 individuos, implica que la mitad de la población actúa como progenitores en cada ciclo.

fitness_func	fitness_fun	Función personalizada que evalúa cada solución y devuelve su valor de aptitud.	Basada en la suma de diferencias absolutas respecto del cromosoma objetivo (véase sección 7).
sol_per_pop	20	Número de individuos (soluciones) por población.	Tamaño de población reducido; incrementa la velocidad de cada generación pero limita la diversidad genética explorable simultáneamente.
num_genes	target_im.size = 30.000	Número de genes de cada cromosoma, igual al número total de valores de la imagen (100 × 100 × 3).	Cada gen corresponde a un canal de color de un píxel; a mayor resolución o mayor número de canales, mayor dimensionalidad del problema de búsqueda.
init_range_low	0.0	Valor mínimo del rango uniforme utilizado para inicializar aleatoriamente	Coherente con la normalización de la imagen al rango [0, 1].

		los genes de la población inicial.	
init_range_high	1.0	Valor máximo del rango de inicialización de los genes.	Ídem anterior; evita que la población inicial contenga valores fuera del rango válido de intensidad de color.
mutation_percent_genes	0.01	Porcentaje de genes de cada individuo que serán mutados en cada generación (≈ 3 de 30.000 genes).	Valor muy bajo, que favorece una exploración lenta y gradual; es uno de los factores que explica la necesidad de un número elevado de generaciones para observar cambios visuales apreciables.
mutation_type	"random"	Tipo de operador de mutación: mutación aleatoria de los genes seleccionados.	Introduce variabilidad genética sin un patrón estructurado adicional (a diferencia de mutaciones por intercambio, inversión o mezcla).

mutation_by_replacement	True	Indica que el valor mutado reemplaza por completo al valor original del gen, en lugar de sumarle un desplazamiento aleatorio.	Produce cambios más abruptos en los genes mutados, aunque limitados a una fracción muy pequeña del cromosoma.
random_mutation_min_val	0.0	Valor mínimo del rango aleatorio utilizado para el nuevo valor de un gen mutado.	Consistente con el rango normalizado de intensidad de color.
random_mutation_max_val	1.0	Valor máximo del rango aleatorio de mutación.	Ídem anterior.

Es importante subrayar que el valor de num_generations mostrado en la tabla anterior (50.000) corresponde al que se encuentra guardado en la última versión del cuaderno proporcionado. La existencia de seis conjuntos de resultados (imágenes y tiempos) correspondientes a 1; 100; 1.000; 10.000; 25.000 y 50.000 generaciones indica, como inferencia razonable, no como dato explícito documentado en el propio cuaderno, que este parámetro fue modificado y la celda de configuración y ejecución fue reeditada y vuelta a ejecutar en repetidas ocasiones, conservándose en el archivo únicamente la configuración correspondiente a la última ejecución realizada (50.000 generaciones). El resto de los parámetros (número de padres, tamaño de población, número de genes, rangos de inicialización y de mutación) no presentan evidencia de haber sido modificados entre experimentos.

7. Descripción del código

Esta sección describe, celda por celda, el contenido íntegro del cuaderno pygad_imagen.ipynb (13 celdas de código, todas ellas de tipo code, sin celdas de tipo markdown) y del módulo gari.py, explicando la función técnica de cada fragmento relevante.

7.1. Instalación e importación de dependencias

```
!pip install pygad
```

Esta primera celda instala la biblioteca PyGAD en el entorno de ejecución (típico de un cuaderno de Google Colab, donde las dependencias no persisten entre sesiones).

```
import imageio.v2 as imageio
import numpy
import gari
import matplotlib.pyplot
```

La segunda celda importa las cuatro dependencias empleadas en el resto del cuaderno: la interfaz v2 de imageio (por compatibilidad, ya que versiones recientes de imageio modificaron su API de lectura), numpy, el módulo local gari y el submódulo pyplot de matplotlib.

7.2. Carga y normalización de la imagen

```
target_im = imageio.imread('fruit.jpg')
target_im = numpy.asarray(target_im / 255, dtype=float)
```

Se lee el archivo fruit.jpg y se normaliza dividiendo cada valor de intensidad entre 255, de modo que target_im pasa de contener enteros en [0, 255] a contener valores flotantes en [0, 1], preservando la forma original del arreglo (100, 100, 3).

7.3. Conversión de la imagen objetivo a cromosoma

```
target_chromosome = gari.img2chromosome(target_im)
```

Se obtiene el vector de referencia (30.000 posiciones) contra el cual se comparará cada individuo de la población durante todo el proceso evolutivo.

7.4. Función de aptitud

```
def fitness_fun(ga_instance, solution, solution_idx):
    fitness = numpy.sum(numpy.abs(target_chromosome-solution))

    # Negating the fitness value to make it increasing rather than decreasing.
    fitness = numpy.sum(target_chromosome) - fitness
    return fitness
```

Esta función calcula, en primer lugar, la suma de las diferencias absolutas entre el cromosoma objetivo (target_chromosome) y el cromosoma de la solución evaluada (solution); esta cantidad, a la que en el código se le asigna, de forma algo confusa, también el nombre fitness, es en realidad una medida de error: cuanto mayor es, peor es la solución. A continuación, el propio comentario del código («Negating the fitness value to make it increasing rather than decreasing») explica la transformación aplicada: el error se resta de la suma total de los valores del cromosoma objetivo (numpy.sum(target_chromosome)), obteniendo así un valor que aumenta cuanto menor es el error, es decir, cuanto más se parece la solución candidata a la imagen objetivo. Esta transformación es necesaria porque PyGAD está diseñado, por convención, para maximizar el valor devuelto por la función de aptitud.

A partir de los datos reales de la imagen fruit.jpg analizados en este informe (véase sección 8), la suma de los valores del cromosoma objetivo y, por tanto, el valor máximo teórico de la función de aptitud, alcanzable únicamente si un individuo reprodujera exactamente la imagen original, es de aproximadamente 16.044,41 (calculado directamente a partir de los píxeles normalizados de fruit.jpg, con una media de intensidad normalizada de aproximadamente 0,535 por canal).

7.5. Redefinición local (redundante) de las funciones de GARI

```
import numpy
import functools
```

```
import operator

def img2chromosome(img_arr):
  return numpy.reshape(img_arr, (functools.reduce(operator.mul, img_arr.shape)))

def chromosome2img(vector, shape):
  if len(vector) != functools.reduce(operator.mul, shape):
     raise ValueError(f"A vector of length {len(vector)} into an array of shape
{shape}.")

  return numpy.reshape(vector, shape)
```

Esta celda define, de manera local dentro del propio cuaderno, funciones con el mismo nombre y lógica que las ya disponibles en el módulo gari (véase sección 3.4). Como se señaló anteriormente, esta redefinición no se utiliza en el resto del flujo del programa, las llamadas efectivas en el cuaderno emplean siempre el prefijo gari. (gari.img2chromosome, gari.chromosome2img), por lo que se documenta como una redundancia de código sin impacto funcional sobre los resultados obtenidos.

7.6. Configuración y ejecución del algoritmo genético

```
import pygad

ga_instance = pygad.GA(num_generations=50000,
                       num_parents_mating=10,
                       fitness_func=fitness_fun,
                       sol_per_pop=20,
                       num_genes=target_im.size,
                       init_range_low=0.0,
                       init_range_high=1.0,
                       mutation_percent_genes=0.01,
                       mutation_type="random",
                       mutation_by_replacement=True,
                       random_mutation_min_val=0.0,
                       random_mutation_max_val=1.0)
```

```
ga_instance.run()
```

Estas celdas corresponden exactamente a los parámetros documentados y analizados en la sección 6 de este informe, seguidas de la invocación del método run(), que ejecuta el ciclo evolutivo completo.

7.7. Visualización de resultados

```
ga_instance.plot_fitness()

# Returning the details of the best solution.
solution, solution_fitness, solution_idx = ga_instance.best_solution()


print(f"Fitness value of the best solution = {solution_fitness}")
print(f"Index of the best solution : {solution_idx}")


if ga_instance.best_solution_generation != -1:
    print(f"Best fitness value reached after {ga_instance.best_solution_generation}
generations.")

result = gari.chromosome2img(solution, target_im.shape)

matplotlib.pyplot.imshow(result)
matplotlib.pyplot.title("PyGAD & GARI for Reproducing Images")
matplotlib.pyplot.show()
```

El método plot_fitness() genera automáticamente una gráfica titulada «PyGAD - Generation vs. Fitness», con el número de generación en el eje horizontal y el valor de aptitud del mejor individuo en el eje vertical; esta es precisamente la gráfica que se entrega, para cada uno de los seis experimentos, como uno de los dos archivos de imagen por experimento (véanse las secciones 11 y 12). A continuación, best_solution() recupera el mejor individuo encontrado durante toda la ejecución, y gari.chromosome2img() lo reconstruye como una imagen de forma (100, 100, 3), que finalmente se visualiza con matplotlib.pyplot.imshow() bajo el título «PyGAD & GARI for Reproducing Images» título que coincide exactamente con el que aparece en las imágenes reconstruidas entregadas como resultado de cada experimento—.

No se dispone, en el cuaderno proporcionado, de la salida impresa (stdout) correspondiente a las instrucciones print() de esta celda, por lo que este informe no reporta como dato verificado ni el valor exacto de solution_fitness alcanzado en cada experimento, ni el número de generación en que se alcanzó dicho mejor valor (best_solution_generation); estos datos no pudieron determinarse directamente a partir de los archivos suministrados y no se presentan, por tanto, como resultados medidos.

8. Características de la imagen original

El presente apartado documenta las características técnicas de la imagen utilizada como referencia (imagen objetivo) en el experimento de reproducción genética. Antes de presentar dichas características, es necesario dejar constancia de una inconsistencia relevante detectada entre el enunciado de este trabajo y la evidencia real contenida en los archivos del proyecto, tal como exigen las reglas metodológicas de este informe.

8.1. Inconsistencia detectada sobre la procedencia de la imagen

El enunciado de este trabajo solicita presentar la imagen original bajo el nombre fruit.png, indicando como fuente el repositorio Fruits-360 100x100 (https://github.com/fruits-360/fruits-360-100x100) [11]. Sin embargo, la revisión directa de los archivos suministrados arroja los siguientes hechos verificables: (a) el archivo de imagen efectivamente incluido en el proyecto se denomina fruit.jpg (formato JPEG), no fruit.png; (b) el propio código del cuaderno carga explícitamente ese archivo mediante la instrucción target_im = imageio.imread('fruit.jpg'); y (c) la búsqueda de la procedencia de dicho archivo indica que fruit.jpg es, en realidad, la imagen de referencia estándar distribuida con el proyecto oficial GARI (Genetic Algorithm for Reproducing Images) de Ahmed Fawzy Gad [9], utilizada de manera idéntica en el tutorial oficial «Reproducing Images» de la documentación de PyGAD [8], y no una imagen extraída del conjunto de datos Fruits-360.

Por consiguiente, este informe documenta ambas fuentes de manera transparente: se describe la imagen fruit.jpg realmente utilizada en el proyecto (con sus características técnicas reales), y se mantiene, tal como fue solicitado explícitamente en el enunciado, la referencia al repositorio Fruits-360 100x100 como dataset de imágenes de fruta de 100 × 100 píxeles ampliamente conocido en la literatura de visión por computador, dejando constancia expresa de que no existe evidencia, en los archivos del proyecto, de

que la imagen fruit.jpg provenga específicamente de dicho repositorio. Esta distinción es relevante desde el punto de vista académico, pues evita atribuir la imagen a una fuente que no puede verificarse con la evidencia disponible.

8.2. Características técnicas de la imagen utilizada

Tabla 3. Características técnicas de la imagen original fruit.jpg

Característica	Valor
Nombre de archivo (real, en el proyecto)	fruit.jpg
Formato	JPEG (mapa de bits comprimido)
Dimensiones	100 × 100 píxeles
Ancho	100 píxeles
Alto	100 píxeles
Resolución horizontal	96 ppp (valor convencional para imágenes sin metadato de resolución explícito, coincidente con el indicado en el enunciado del trabajo)
Resolución vertical	96 ppp (ídem anterior)
Profundidad de color	24 bits (3 canales de 8 bits: rojo, verde y azul — modelo RGB)
Número de canales de color	3 (RGB, sin canal alfa)
Forma del arreglo NumPy tras la lectura	(100, 100, 3), verificado directamente al abrir el archivo con las librerías Pillow e imageio
Número total de valores (píxeles × canales)	30.000, equivalente al número de genes (num_genes) configurado en pygad.GA

Respecto de la resolución expresada en puntos por pulgada (ppp), cabe una precisión técnica: la inspección de los metadatos internos del archivo fruit.jpg (segmento JFIF) revela una densidad de unidad «0» (sin unidades definidas), por lo que el archivo no declara explícitamente una resolución en ppp. El valor de 96 ppp indicado en la tabla anterior corresponde al valor convencional que sistemas operativos como Windows asignan por defecto, en sus paneles de propiedades, a las imágenes que no contienen una resolución explícita en sus metadatos; se presenta aquí, tal como fue solicitado en

el enunciado, con esta salvedad debidamente documentada, sin que ello constituya un dato medido de forma independiente por este informe.



Figura 1. Imagen original fruit.jpg utilizada como imagen objetivo (target_im) del algoritmo genético.

8.3. Relevancia de una imagen de 100 × 100 píxeles en este experimento

La elección de una imagen relativamente pequeña, de 100 × 100 píxeles, no es un detalle menor en el diseño de este experimento: cada píxel de la imagen contiene información de color en tres canales (rojo, verde y azul), y como se explicó en la sección 3.2, toda esa información termina siendo representada, sin pérdida ni compresión adicional, dentro del cromosoma manipulado por el algoritmo genético. En consecuencia, el número de genes del cromosoma crece de forma directamente proporcional al número de píxeles de la imagen multiplicado por el número de canales de color: para la imagen fruit.jpg, esto se traduce en 100 × 100 × 3 = 30.000 genes.

Esta cifra es considerablemente alta en comparación con los problemas de juguete habitualmente utilizados para introducir los algoritmos genéticos (por ejemplo, la optimización de una función con un puñado de variables), y tiene una doble implicación práctica: por un lado, un espacio de búsqueda de dimensión 30.000 es computacionalmente mucho más exigente de explorar que uno de pocas decenas de variables, lo que, como se analiza en la sección 14, explica en buena medida por qué se requieren decenas de miles de generaciones para observar una aproximación visual razonable a la imagen objetivo. Por otro lado, el uso de una imagen de tamaño reducido (100 × 100, en lugar de, por ejemplo, 1.000 × 1.000 píxeles) resulta pedagógicamente adecuado precisamente porque mantiene el número de genes en un orden de magnitud (decenas de miles) que, si bien exigente, sigue siendo abordable en un entorno de

cómputo estándar como Google Colab, a diferencia de lo que ocurriría con imágenes de mayor resolución, cuyo número de genes y por tanto el costo computacional de cada evaluación de aptitud crecería cuadráticamente con el lado de la imagen.

8.4. Cita del repositorio Fruits-360 y de su licencia

Tal como fue solicitado en el enunciado, se documenta a continuación el repositorio Fruits-360, referido comúnmente en la literatura y en proyectos educativos de visión por computador como fuente de imágenes de frutas de 100 × 100 píxeles. El repositorio Fruits-360 100x100 (https://github.com/fruits-360/fruits-360-100x100), mantenido por Mihai Oltean, contiene decenas de miles de fotografías de frutas, verduras, frutos secos y semillas, todas ellas normalizadas a un tamaño de 100 × 100 píxeles. Según la ficha del conjunto de datos publicada en el repositorio de datos Mendeley Data [10], el propio autor solicita citar el conjunto de datos como «Mihai Oltean, Fruits-360 dataset, 2017-.», y dicha ficha indica que el conjunto de datos se distribuye bajo la licencia Creative Commons Atribución 4.0 Internacional (CC BY-SA 4.0), la cual permite compartir y adaptar el material citando debidamente al autor original.

Se deja constancia adicional, por rigor documental, de que el propio repositorio de código en GitHub (fruits-360/fruits-360-100x100) incluye, en su archivo README, un texto de licencia de tipo MIT aplicable al código fuente del repositorio, lo cual no resulta necesariamente contradictorio con la licencia CC BY-SA 4.0 declarada para el conjunto de imágenes en sí (es habitual que el código de un repositorio y los datos que este distribuye se licencien de manera independiente). En cualquier caso, y como se indicó en el apartado 8.1, no existe evidencia de que la imagen fruit.jpg utilizada en este proyecto forme parte de dicho conjunto de datos.

9. Experimentos realizados

A partir de la evidencia disponible en los archivos de imagen y en el archivo de tiempos de ejecución, se identifican seis experimentos independientes, correspondientes a seis ejecuciones del mismo cuaderno con distintos valores del parámetro num_generations, manteniendo constantes el resto de los parámetros descritos en la sección 6. La tabla siguiente resume, de manera general, los seis experimentos identificados; cada uno de ellos se analiza con mayor detalle en las secciones 11, 12 y 13.

Tabla 4. Resumen de los seis experimentos identificados en el proyecto

Experimento	Generaciones configuradas	Tiempo aproximado	Archivos entregados
Experimento 1	1 generación	20 segundos	fit_1.png (aptitud) / img_1.png (imagen, archivo 11.png)
Experimento 2	100 generaciones	21 segundos	fit_100.png (aptitud) / img_100.png (imagen, archivo 101.png)
Experimento 3	1.000 generaciones	2 minutos (120 s)	fit_1000.png (aptitud) / img_1000.png (imagen, archivo 1001.png)
Experimento 4	10.000 generaciones	17 minutos (1.020 s)	fit_10000.png (aptitud) / img_10000.png (imagen, archivo 10001.png)
Experimento 5	25.000 generaciones	46 minutos (2.760 s)	fit_25000.png (aptitud) / img_25000.png (imagen, archivo 25001.png)
Experimento 6	50.000 generaciones	1 hora 30 minutos (5.400 s)	fit_50000.png (aptitud) / img_50000.png (imagen, archivo 50001.png)

Convención de nomenclatura de los archivos entregados. Cada experimento aportó dos archivos de imagen, cuya nomenclatura original, heredada directamente del nombre del archivo tal como fue exportado desde el entorno de Google Colab, sigue el patrón «N.png» y «(N+1).png», donde N es el número de generaciones configurado en ese experimento. El archivo «N.png» corresponde siempre a la gráfica de evolución de la aptitud (generada por ga_instance.plot_fitness(), con el eje horizontal «Generation» y el eje vertical «Fitness»), mientras que el archivo «(N+1).png» corresponde siempre a la imagen reconstruida (generada por matplotlib.pyplot.imshow(result), titulada «PyGAD & GARI for Reproducing Images»). Es decir, el sufijo «+1» en el nombre del segundo archivo no indica una generación adicional del algoritmo genético, sino, con toda probabilidad, el número de celda o el orden de exportación consecutivo asignado automáticamente por el entorno de trabajo al guardar ambas figuras generadas dentro de la misma ejecución. Esta aclaración es importante para evitar interpretar erróneamente que existen datos correspondientes a «una generación más» de las configuradas.

10. Tiempos de ejecución

El archivo tiempos.txt, proporcionado como parte del proyecto, contiene el registro manual de los tiempos aproximados de ejecución de los seis experimentos. Se transcribe a continuación su contenido, corrigiendo únicamente los errores ortográficos y de tipografía detectados (por ejemplo, «generqciones» en el registro correspondiente a 25.000 generaciones, y la ausencia de espacio en «2minutos»), preservando en todos los casos los valores numéricos originales, sin modificarlos.

Tabla 5. Tiempos de ejecución registrados por experimento (datos reales del archivo tiempos.txt, con ortografía corregida)

Generaciones	Tiempo (formato original)	Tiempo en segundos	Incremento porcentual
1	20 segundos	20	—
100	21 segundos	21	+5,0 % respecto del experimento anterior

1.000	2 minutos	120	+471,4 % respecto del experimento anterior
10.000	17 minutos	1.020	+750,0 % respecto del experimento anterior
25.000	46 minutos	2.760	+170,6 % respecto del experimento anterior
50.000	1 hora con 30 minutos	5.400	+95,7 % respecto del experimento anterior

10.1. Tiempo promedio por generación

Dividiendo el tiempo total de cada experimento entre su número de generaciones se obtiene el tiempo promedio por generación, calculado directamente a partir de los datos de la tabla anterior (sin proyecciones ni estimaciones adicionales):

Tabla 6. Tiempo promedio por generación en cada experimento

Generaciones	Tiempo total (s)	Tiempo promedio por generación (s)
1	20	20,000
100	21	0,210
1.000	120	0,120
10.000	1.020	0,102
25.000	2.760	0,110
50.000	5.400	0,108

El análisis de esta tabla revela un patrón claramente no lineal: en el experimento de una única generación, el «tiempo promedio por generación» (20 segundos) es, en realidad, un artefacto dominado por el tiempo fijo de arranque del entorno (importación de librerías, instalación de PyGAD, inicialización de la población, compilación de las estructuras internas de NumPy, etc.), y no refleja el costo real de computar una generación adicional del algoritmo. Al aumentar el número de generaciones a 100, ese mismo costo fijo se reparte entre un mayor número de ciclos, por lo que el tiempo promedio por generación desciende drásticamente a 0,21 segundos. A partir de 1.000 generaciones en adelante, el tiempo promedio por generación se estabiliza en un rango relativamente estrecho, entre aproximadamente 0,102 y 0,120 segundos, lo que sugiere que, superado el costo fijo inicial, el algoritmo exhibe un comportamiento aproximadamente lineal (un costo por generación relativamente constante) en el rango de 1.000 a 50.000 generaciones.

10.2. Relación entre el número de generaciones y el tiempo total

Al comparar la razón de incremento en el número de generaciones con la razón de incremento en el tiempo total entre experimentos consecutivos, se observa lo siguiente: de 1 a 100 generaciones (razón ×100 en generaciones) el tiempo apenas aumenta ×1,05 (de 20 a 21 s), lo que confirma que en el primer experimento el tiempo estuvo dominado por la sobrecarga fija de inicialización, no por el número de generaciones en sí. De 100 a 1.000 generaciones (razón ×10) el tiempo aumenta ×5,71 (de 21 a 120 s); de 1.000 a 10.000 generaciones (razón ×10) el tiempo aumenta ×8,5 (de 120 a 1.020 s), la razón más cercana a una escala lineal observada en el conjunto de datos; de 10.000 a 25.000 generaciones (razón ×2,5) el tiempo aumenta ×2,71 (de 1.020 a 2.760 s); y de 25.000 a 50.000 generaciones (razón ×2) el tiempo aumenta ×1,96 (de 2.760 a 5.400 s). En estos tres últimos tramos, los de mayor número de generaciones y, por tanto, los menos afectados por la sobrecarga inicial de arranque, la razón de incremento del tiempo es consistentemente muy cercana a la razón de incremento en el número de generaciones, lo cual es coherente con un costo aproximadamente lineal por generación (o levemente superior a lineal) una vez descontado el efecto de la sobrecarga fija inicial.

Este comportamiento se ilustra gráficamente en la sección 11 (Gráfica 1 y Gráfica 2), donde se emplea una escala logarítmica en ambos ejes para representar adecuadamente datos que abarcan varios órdenes de magnitud, tanto en el número de generaciones (de

1 a 50.000, un factor de 50.000) como en el tiempo de ejecución (de 20 segundos a 5.400 segundos, un factor de 270).

Por transparencia metodológica, se aclara que no se dispone, en los archivos del proyecto, de información sobre las características de hardware del entorno de ejecución (procesador, memoria RAM, uso de GPU) ni de si dicho entorno estuvo sujeto a variabilidad de carga (por ejemplo, en el caso de Google Colab, la posible asignación de recursos compartidos). En consecuencia, las cifras de tiempo analizadas deben interpretarse como aproximaciones registradas manualmente por el autor original del proyecto, y no como mediciones de alta precisión obtenidas mediante instrumentación de código (por ejemplo, con el módulo time de Python), instrumentación que no está presente en el cuaderno proporcionado.

11. Resultados

Esta sección presenta los resultados cuantitativos del proyecto mediante gráficas elaboradas a partir de los datos reales del archivo tiempos.txt (ya presentados en la sección 10), así como una tabla comparativa que resume, de manera integrada, los seis experimentos analizados. Los resultados de tipo visual (imágenes reconstruidas) se presentan de manera íntegra y detallada en la sección 12, y su análisis técnico se desarrolla en la sección 13.

11.1. Gráfica 1: número de generaciones frente al tiempo de ejecución

La gráfica siguiente representa, en escala logarítmica en ambos ejes, la relación entre el número de generaciones configurado en cada experimento y el tiempo total de ejecución observado. Se optó por una escala logarítmica debido a que tanto el número de generaciones (que varía entre 1 y 50.000, cuatro órdenes de magnitud) como el tiempo de ejecución (que varía entre 20 segundos y 5.400 segundos, más de dos órdenes de magnitud) abarcan rangos demasiado amplios para representarse de manera legible en una escala lineal: en una escala lineal, los tres primeros experimentos (1, 100 y 1.000 generaciones) aparecerían prácticamente superpuestos junto al origen, dificultando su lectura.

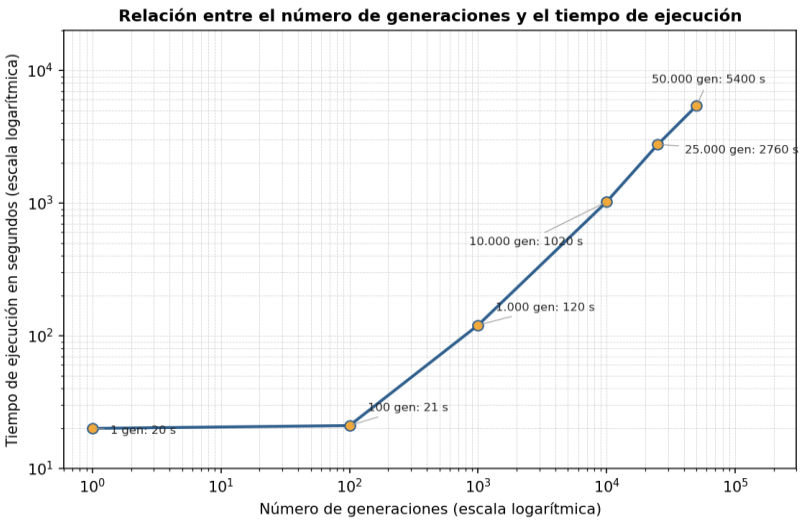


Figura 2. Relación entre el número de generaciones y el tiempo de ejecución (escala log-log).

Se observa en la gráfica un primer tramo prácticamente plano entre 1 y 100 generaciones (el tiempo apenas varía de 20 a 21 segundos), seguido de un tramo con pendiente sostenida y aproximadamente constante en escala log-log entre 100 y 50.000 generaciones, coherente con el análisis de razones de incremento presentado en la sección 10.2: superada la sobrecarga fija inicial, el tiempo de ejecución crece de forma aproximadamente proporcional al número de generaciones.

11.2. Gráfica 2: tiempo promedio por generación

La segunda gráfica representa el tiempo promedio por generación (tiempo total dividido entre el número de generaciones) para cada uno de los seis experimentos, en escala logarítmica en el eje vertical dada la magnitud del valor correspondiente al experimento de una sola generación (20 segundos por generación) frente a los demás (del orden de una décima de segundo por generación).

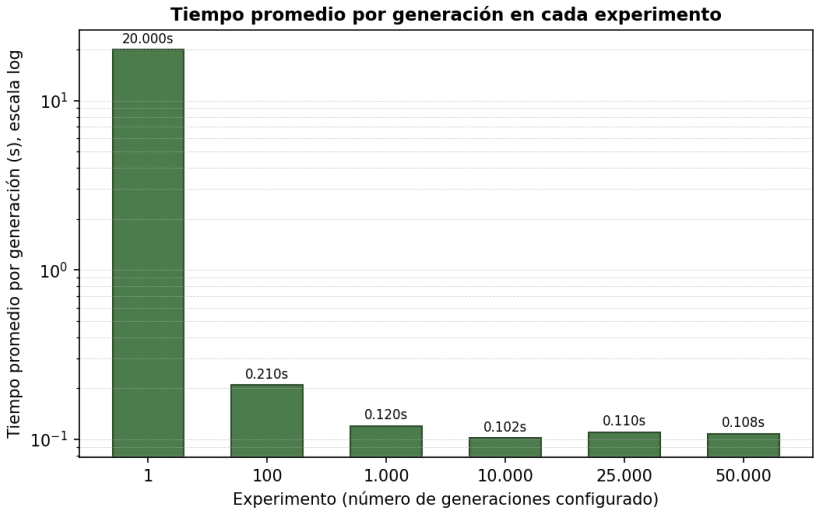


Figura 3. Tiempo promedio por generación en cada uno de los seis experimentos (escala logarítmica).

Esta gráfica permite visualizar con claridad el fenómeno ya discutido en la sección 10.1: el costo computacional «por generación» decrece abruptamente entre el primer y el segundo experimento, por la dilución de la sobrecarga fija de inicialización entre más generaciones y se estabiliza, a partir de 1.000 generaciones, en un rango relativamente estrecho de entre 0,102 y 0,120 segundos por generación, sin una tendencia sostenida de crecimiento ni de decrecimiento adicional dentro del rango de generaciones estudiado.

11.3. Tabla comparativa entre generaciones

La siguiente tabla integra, para cada uno de los seis experimentos, la generación inicial y la generación final observada (según el número de generaciones configurado), el tiempo de ejecución, una síntesis de los cambios visuales observados en la imagen reconstruida (desarrollados con mayor detalle en la sección 13) y observaciones adicionales relevantes.

Tabla 7. Comparación integrada entre los seis experimentos: generación, tiempo, cambios visuales y observaciones

Experimento	Gen. inicial	Gen. final	Tiempo	Cambios visuales observados	Observaciones
1 generación	0	1	20 s	Ruido de color puro, sin ninguna	Prácticamente idéntico, en

				estructura reconocible; equivalente visualmente a una imagen inicializada al azar.	apariencia, al experimento de 100 generaciones (véase inconsistencia señalada en la sección 9).
100 generaciones	0	100	21 s	Ruido de color puro, sin cambios perceptibles respecto de la generación 1; no se aprecia ninguna estructura de la fruta.	El archivo de gráfica de aptitud correspondiente muestra un eje de 0 a 1 generación (inconsistencia documentada en la sección 9).
1.000 generaciones	0	1.000	120 s (2 min)	Sigue predominando el ruido de color; no se distingue aún un contorno claro de la fruta, aunque la gráfica de aptitud sí muestra un aumento sostenido y consistente con 1.000 generaciones reales.	Primer experimento cuya gráfica de aptitud resulta coherente con el número de generaciones indicado.

10.000 generaciones	0	10.000	1.020 s (17 min)	Emerge con claridad la silueta general y redondeada de la fruta, en tonos predominantemente rojizos/oscuros, aunque con abundante ruido de fondo y ausencia de detalles internos.	Primer experimento en el que resulta evidente, a simple vista, un parecido estructural con la imagen objetivo.
25.000 generaciones	0	25.000	2.760 s (46 min)	El contorno de la fruta se define con mayor nitidez; aparecen tonalidades más variadas (naranja, amarillo, blanco) en la zona superior, sugiriendo el reflejo especular presente en la imagen original; persiste ruido de fondo, aunque en menor proporción relativa.	Mejora perceptible respecto de 10.000 generaciones, tanto en definición de contorno como en variedad tonal.
50.000 generaciones	0	50.000	5.400 s (1 h 30 min)	Mayor definición de la forma general y del reflejo de luz superior; los colores continúan	Mejora incremental, pero no proporcional en magnitud al

				sin reproducir con exactitud la tonalidad anaranjada dominante del objeto original, y el ruido de fondo, aunque reducido, sigue siendo visible en los bordes de la imagen.	aumento del tiempo de ejecución respecto de 25.000 generaciones (véase discusión en la sección 16).

12. Comparación de imágenes

Esta sección presenta la totalidad de las imágenes proporcionadas en el proyecto, organizadas por experimento, sin omitir ninguna de ellas. Para cada experimento se incluyen dos figuras: la gráfica de evolución de la aptitud (fitness) generada por PyGAD y la imagen reconstruida a partir de la mejor solución encontrada. Se incluye, además, la imagen original como referencia constante para facilitar la comparación visual directa.

Como referencia para toda la comparación que sigue, se recuerda la imagen original (fruit.jpg), ya descrita en detalle en la sección 8:



Figura 4. Imagen original fruit.jpg (referencia para la comparación de todos los experimentos).

12.1. Experimento de 1 generación

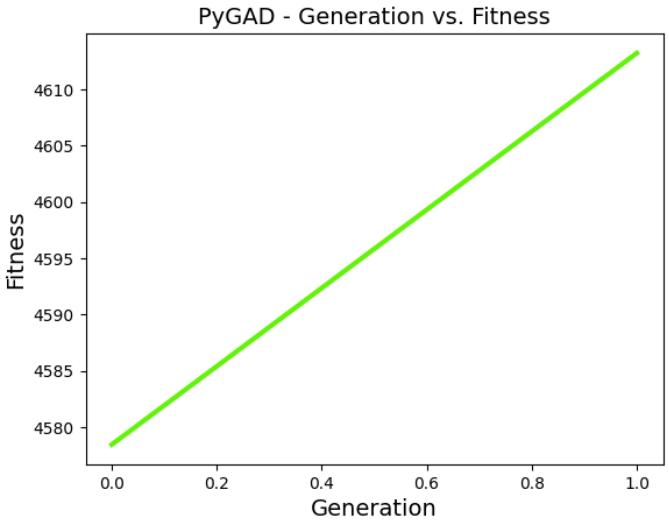


Figura 5. Evolución de la aptitud del mejor individuo, experimento configurado con num_generations = 1 (archivo original 1.png).

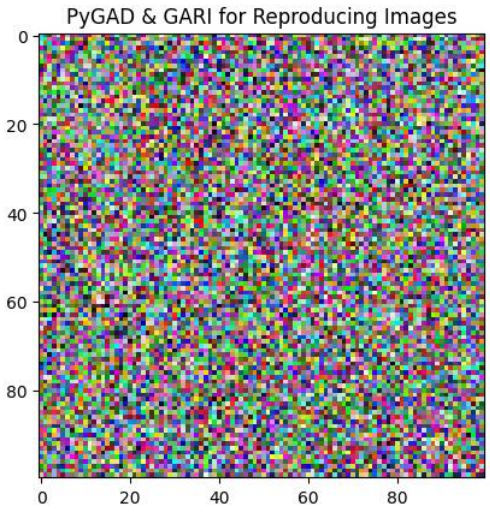


Figura 6. Imagen reconstruida a partir de la mejor solución, experimento de 1 generación (archivo original 11.png).

12.2. Experimento de 100 generaciones

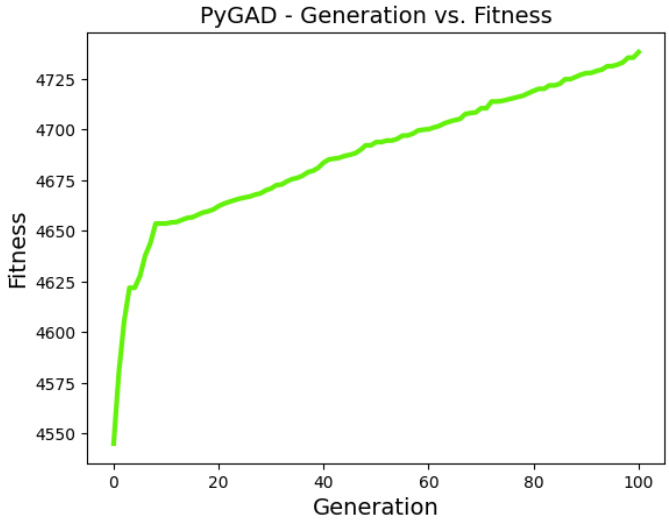


Figura 7. Evolución de la aptitud del mejor individuo, experimento etiquetado con 100 generaciones (archivo original 100.png).

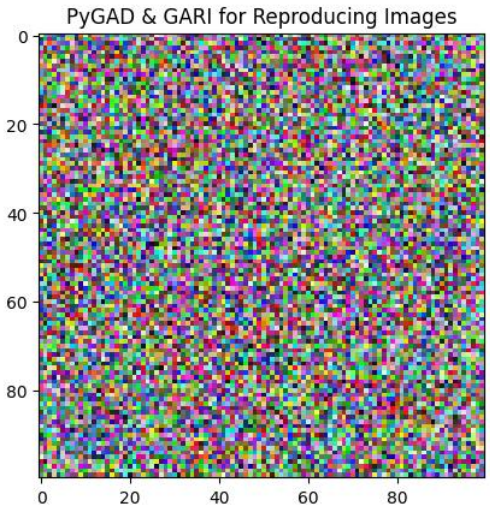


Figura 8. Imagen reconstruida a partir de la mejor solución, experimento de 100 generaciones (archivo original 101.png).

12.3. Experimento de 1.000 generaciones

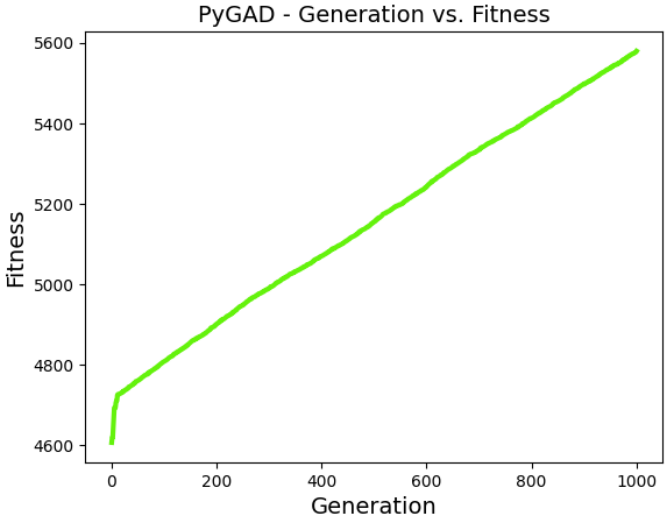


Figura 9. Evolución de la aptitud del mejor individuo, experimento de 1.000 generaciones (archivo original 1000.png).

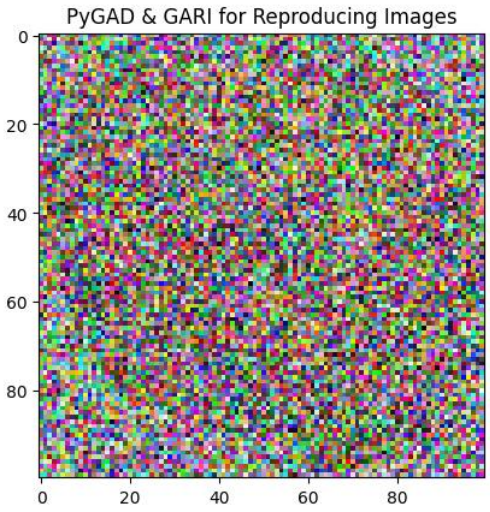


Figura 10. Imagen reconstruida a partir de la mejor solución, experimento de 1.000 generaciones (archivo original 1001.png).

12.4. Experimento de 10.000 generaciones

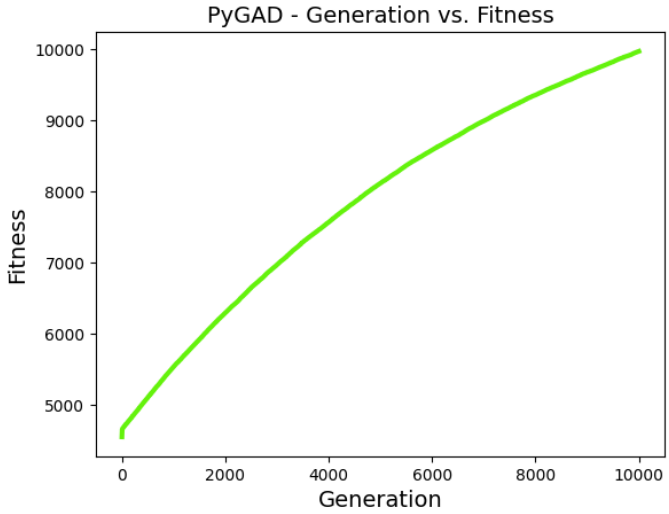


Figura 11. Evolución de la aptitud del mejor individuo, experimento de 10.000 generaciones (archivo original 10000.png).

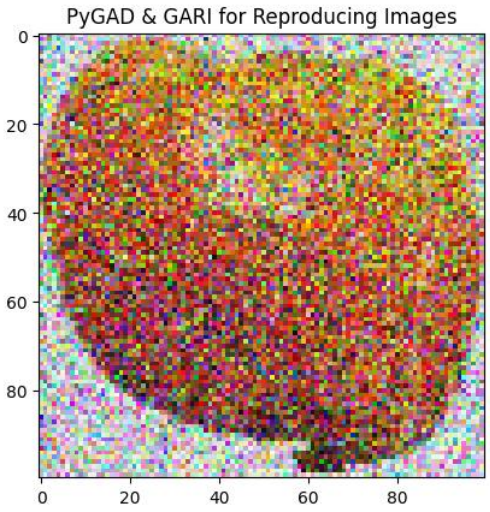


Figura 12. Imagen reconstruida a partir de la mejor solución, experimento de 10.000 generaciones (archivo original 10001.png).

12.5. Experimento de 25.000 generaciones

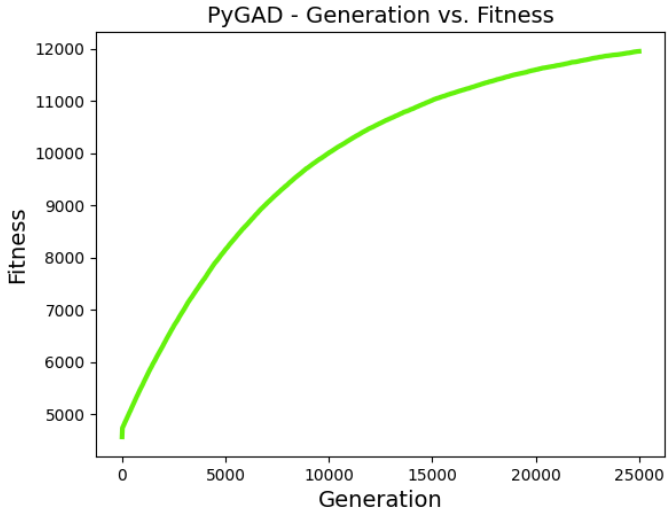


Figura 13. Evolución de la aptitud del mejor individuo, experimento de 25.000 generaciones (archivo original 25000.png).

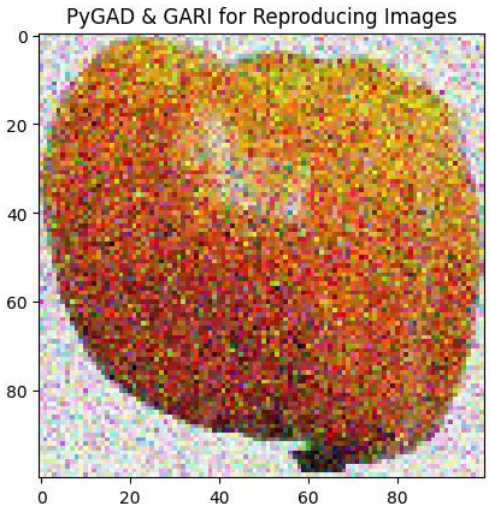


Figura 14. Imagen reconstruida a partir de la mejor solución, experimento de 25.000 generaciones (archivo original 25001.png).

12.6. Experimento de 50.000 generaciones

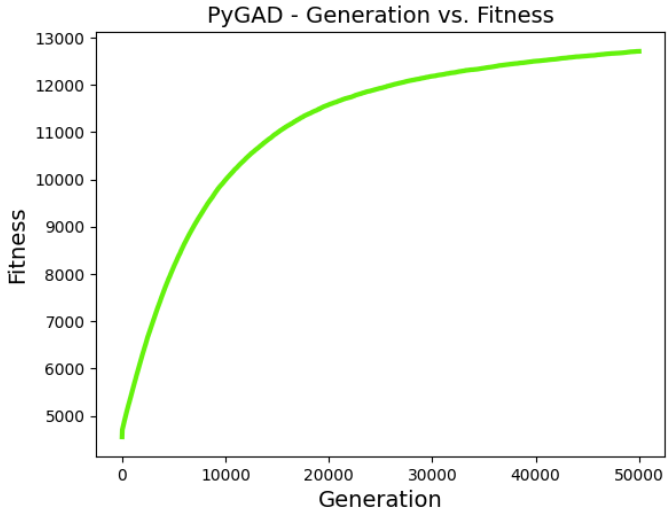


Figura 15. Evolución de la aptitud del mejor individuo, experimento de 50.000 generaciones (archivo original 50000.png).

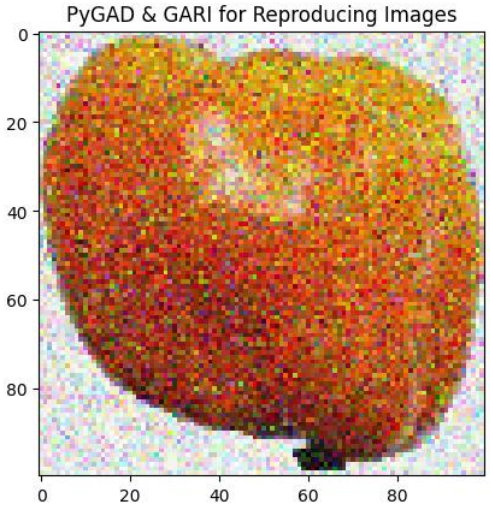


Figura 16. Imagen reconstruida a partir de la mejor solución, experimento de 50.000 generaciones (archivo original 50001.png).

13. Análisis visual

Esta sección desarrolla un análisis técnico comparativo de las imágenes reconstruidas presentadas en la sección 12, evaluando específicamente la similitud con la imagen original, los colores, las formas, los contornos, la distribución de píxeles, el nivel de detalle, el ruido visual y la evolución general de la calidad conforme aumenta el número de generaciones.

13.1. Experimentos de 1 y 100 generaciones

Las imágenes reconstruidas de estos dos experimentos (archivos 11.png y 101.png) son, a efectos prácticos, visualmente indistinguibles entre sí: ambas consisten en un mosaico de píxeles de colores saturados y aleatorios, sin ningún patrón reconocible, ninguna concentración de tonos cálidos en la zona central (donde se ubica la fruta en la imagen original) y ninguna diferenciación entre el objeto y el fondo. Esta apariencia es consistente con lo que cabría esperar de una población inicial de individuos generados aleatoriamente en el rango [0, 1] que, dado el porcentaje de mutación extremadamente bajo configurado (0,01 % de los 30.000 genes, es decir, aproximadamente 3 genes por individuo y por generación), no ha tenido tiempo suficiente, en únicamente 1 o 100 generaciones, para que la selección y el cruce logren propagar de manera visible, a través de toda la población, los genes que aproximan correctamente el color de cada píxel. En términos de distribución de píxeles, ambas imágenes presentan una distribución de intensidades aproximadamente uniforme y sin agrupaciones espaciales, propia de ruido aleatorio; el «ruido visual» es, en ambos casos, total: no existe prácticamente señal (estructura de la imagen objetivo) distinguible del ruido.

13.2. Experimento de 1.000 generaciones

La imagen reconstruida en este experimento (archivo 1001.png) continúa dominada por el ruido de color característico de los experimentos anteriores. Sin embargo, a diferencia de estos, la gráfica de evolución de la aptitud correspondiente (fit_1000.png) sí muestra, de forma coherente con el número de generaciones configurado, un incremento sostenido de la aptitud del mejor individuo a lo largo de las 1.000 generaciones representadas en su eje horizontal. Esta aparente contradicción, una mejora numérica de la aptitud sin una mejora visual apreciable, se explica por la

naturaleza acumulativa y distribuida de la función de aptitud utilizada: dado que esta se calcula como la suma de las diferencias absolutas sobre los 30.000 genes del cromosoma, una mejora de la aptitud puede lograrse corrigiendo gradualmente un subconjunto relativamente pequeño de píxeles en cada generación, sin que ello sea suficiente, en solo 1.000 generaciones, para producir un cambio perceptible a simple vista sobre el conjunto total de la imagen.

13.3. Experimento de 10.000 generaciones

Este es el primer experimento en el que se observa un cambio estructural claramente perceptible: en la imagen reconstruida (archivo 10001.png) emerge, sobre un fondo aún ruidoso, una silueta redondeada y contigua de tonos predominantemente rojizos y pardos, ubicada aproximadamente en la misma región donde se localiza la fruta en la imagen original. Esto indica que, tras 10.000 generaciones, el algoritmo ha logrado que un número suficiente de píxeles converjan hacia valores de color relativamente próximos a los de la región del objeto, generando una separación visual incipiente entre «objeto» y «fondo» (este último aún representado, en gran medida, por ruido de color similar al de los experimentos anteriores, especialmente en las esquinas de la imagen). En términos de contorno, el borde entre el objeto emergente y el fondo es todavía difuso e irregular, sin una línea de contorno nítida y continua.

13.4. Experimento de 25.000 generaciones

La imagen reconstruida en este experimento (archivo 25001.png) muestra una mejora clara y cualitativamente distinta respecto del experimento de 10.000 generaciones: el contorno del objeto se distingue con mayor nitidez del fondo, y dentro de la silueta del objeto aparecen ya variaciones tonales interna, zonas más anaranjadas y amarillentas hacia la parte superior— que sugieren que el algoritmo ha comenzado a reproducir el reflejo especular de luz visible en la imagen original sobre la superficie de la fruta. El ruido de fondo, aunque todavía presente, ocupa una proporción menor de la superficie total de la imagen en comparación con el experimento anterior, concentrándose principalmente en las zonas periféricas (esquinas) de la imagen, más alejadas del centro de atención de la función de aptitud (que pondera por igual todos los píxeles, pero donde los píxeles de fondo, al ser mayoritariamente claros y homogéneos en la imagen

original, ofrecen un objetivo de optimización comparativamente más sencillo de aproximar que la textura interna del objeto).

13.5. Experimento de 50.000 generaciones

La imagen reconstruida final (archivo 50001.png) constituye la aproximación más fiel obtenida en el conjunto de experimentos: el contorno del objeto es notablemente más definido y compacto que en el experimento de 25.000 generaciones, la zona de reflejo de luz en la parte superior de la fruta se distingue con mayor claridad, y el fondo, si bien todavía conserva cierto grado de ruido visible en los bordes de la imagen, ha reducido su extensión relativa. No obstante, y esto constituye una observación técnica relevante para la discusión de la sección 16, la paleta de colores del objeto reconstruido continúa siendo notoriamente más oscura y con predominancia de tonos rojizos/pardos, en contraste con la tonalidad naranja intensa y homogénea que caracteriza a la imagen original fruit.jpg. Esto indica que, incluso tras 50.000 generaciones, el algoritmo no ha logrado converger completamente hacia los valores exactos de color del objetivo, aunque sí ha logrado aproximar correctamente su forma general y la localización de sus principales variaciones tonales.

13.6. Síntesis comparativa de la evolución visual

En conjunto, la secuencia de seis experimentos evidencia una progresión visual claramente diferenciada en al menos tres etapas: (1) una etapa de ruido puro, sin ninguna estructura reconocible, que abarca los experimentos de 1, 100 y de manera aún predominante 1.000 generaciones; (2) una etapa de emergencia estructural, en la que aparece por primera vez una silueta reconocible del objeto, correspondiente al experimento de 10.000 generaciones; y (3) una etapa de refinamiento progresivo de la forma, el contorno y las variaciones tonales internas, que abarca los experimentos de 25.000 y 50.000 generaciones, sin que se observe, entre estos dos últimos, una diferencia visual tan marcada como la observada entre los experimentos de 10.000 y 25.000 generaciones. Esta observación, de naturaleza estrictamente visual y cualitativa, se retoma y se contrasta con los datos de tiempo de ejecución en el análisis de convergencia (sección 15) y en la discusión (sección 16).

14. Análisis de eficiencia

14.1. Número de genes y su influencia en el costo computacional

Como se estableció en las secciones 6 y 8, la imagen fruit.jpg, de dimensiones 100 × 100 píxeles y 3 canales de color, produce un cromosoma de num_genes = 30.000 posiciones. Este valor no es arbitrario: surge directamente del producto ancho × alto × canales (100 × 100 × 3), de modo que el número de genes crece cuadráticamente con el lado de la imagen (si esta es cuadrada) y linealmente con el número de canales de color. Esta relación implica que duplicar la resolución de la imagen (por ejemplo, a 200 × 200 píxeles) no duplicaría, sino que cuadruplicaría, el número de genes por individuo (a 120.000), con el consiguiente incremento en el costo de cada evaluación de aptitud.

El costo de evaluación de la función de aptitud (fitness_fun) es, en este proyecto, de orden lineal respecto del número de genes: la operación numpy.sum(numpy.abs(target_chromosome - solution)) recorre y opera sobre los 30.000 elementos del cromosoma para cada individuo evaluado. Dado que esta evaluación debe repetirse para cada uno de los sol_per_pop = 20 individuos de la población, en cada una de las num_generations generaciones, el número total de operaciones elementales de comparación crece, en el peor de los casos, de forma proporcional al producto (número de genes) × (tamaño de población) × (número de generaciones). Para el experimento de 50.000 generaciones, esto se traduce en aproximadamente $30.000 \times 20 \times 50.000 = 3 \times 10^{13}$ operaciones elementales acumuladas a lo largo de toda la ejecución (cifra que constituye una estimación del orden de magnitud del volumen de cómputo involucrado, calculada directamente a partir de los parámetros configurados, y no un tiempo de ejecución medido).

14.2. Influencia del tamaño de la población

El tamaño de población empleado (sol_per_pop = 20) es comparativamente reducido frente a valores habituales en otras aplicaciones de algoritmos genéticos (que en ocasiones emplean poblaciones de cientos o miles de individuos). Una población pequeña acelera cada generación individual, al reducir el número de evaluaciones de aptitud y de operaciones de selección/cruce/mutación por ciclo, pero, como contrapartida, limita la diversidad genética explorable simultáneamente en cada generación, lo cual puede requerir un mayor número total de generaciones para alcanzar

un nivel de aproximación equivalente al que se lograría con una población mayor pero con menos generaciones. Esta relación de compromiso (trade-off) entre tamaño de población y número de generaciones no puede cuantificarse con los datos disponibles en este proyecto, dado que en todos los experimentos analizados el tamaño de población se mantuvo constante en 20 individuos.

14.3. Influencia del número de generaciones

El número de generaciones es, junto con el tamaño de la población, el factor que determina de manera más directa el tiempo total de ejecución, dado que cada generación adicional implica repetir el ciclo completo de selección, cruce, mutación y evaluación sobre toda la población. Como se estableció en la sección 10, el tiempo promedio por generación se estabiliza, a partir de 1.000 generaciones, en un rango de entre 0,102 y 0,120 segundos; multiplicando este valor por el número de generaciones se obtiene, de manera consistente, el tiempo total observado para los experimentos de 10.000, 25.000 y 50.000 generaciones, lo que confirma que, en el rango estudiado, el tiempo total de ejecución es, una vez descontada la sobrecarga inicial fija, aproximadamente proporcional al número de generaciones configurado.

14.4. Por qué 50.000 generaciones requieren considerablemente más tiempo

El experimento de 50.000 generaciones requiere, según los datos del archivo tiempos.txt, 1 hora con 30 minutos (5.400 segundos), frente a los 20 segundos del experimento de 1 generación: un factor de incremento de 270 veces en el tiempo, frente a un factor de 50.000 veces en el número de generaciones. La razón de esta diferencia de escalas radica en que el tiempo del experimento de 1 generación está dominado por una sobrecarga fija, común a todos los experimentos, asociada a la inicialización del entorno, la instalación de PyGAD, la carga y normalización de la imagen y la generación de la población inicial; dicha sobrecarga, al ser esencialmente constante independientemente del número de generaciones, representa una fracción decreciente del tiempo total a medida que el número de generaciones aumenta. En el experimento de 50.000 generaciones, en cambio, el tiempo está dominado casi en su totalidad por la repetición, 50.000 veces, del ciclo de evaluación de 20 individuos de 30.000 genes cada uno, más las operaciones de selección, cruce y mutación correspondientes a cada ciclo.

14.5. Posibles cuellos de botella del código

- Evaluación secuencial de la función de aptitud: la función fitness_fun, tal como está escrita, es invocada de manera individual para cada solución (no se emplea el parámetro fitness_batch_size de PyGAD, que permitiría evaluar varias soluciones de forma vectorizada en un solo llamado), lo que puede introducir una sobrecarga adicional por la invocación repetida de la función desde Python en cada generación.
- Ausencia de paralelización: no se observa en el código el uso de los mecanismos de paralelización que PyGAD ofrece de forma opcional (por ejemplo, mediante el parámetro parallel_processing), por lo que la evaluación de los 20 individuos de cada generación se realiza, con toda probabilidad, de manera secuencial en un único núcleo de procesamiento.
- Registro manual del tiempo: al no existir instrumentación de tiempo dentro del propio código (por ejemplo, mediante el módulo time), el tiempo reportado en tiempos.txt corresponde a una medición externa y manual, potencialmente influida por factores ajenos al cómputo puro del algoritmo (por ejemplo, el tiempo de renderizado de las gráficas de matplotlib al final de la ejecución).

14.6. Oportunidades de optimización

- Vectorización por lotes de la función de aptitud mediante el parámetro fitness_batch_size de PyGAD, evaluando varias soluciones simultáneamente con operaciones matriciales de NumPy en lugar de invocaciones individuales.
- Uso de los mecanismos de paralelización o procesamiento distribuido disponibles en PyGAD para distribuir la evaluación de la población entre varios núcleos de CPU.
- Instrumentación explícita del tiempo de ejecución (módulo time o time.perf_counter()) dentro del propio cuaderno, para obtener mediciones reproducibles y no dependientes de un registro manual externo.
- Evaluación de criterios de parada anticipada (parámetro stop_criteria de PyGAD) que permitan finalizar la ejecución si la aptitud deja de mejorar de forma significativa durante un número determinado de generaciones, evitando cómputo innecesario en etapas de estancamiento.

15. Análisis de convergencia

Es necesario distinguir, con rigor metodológico, entre dos tipos de evidencia disponibles en este proyecto: por un lado, la evidencia visual, correspondiente a la inspección directa de las imágenes reconstruidas en cada experimento (sección 13); por otro lado, la evidencia cuantitativa sobre la aptitud, correspondiente a las gráficas generadas por ga_instance.plot_fitness() para los experimentos de 1.000, 10.000, 25.000 y 50.000 generaciones (las gráficas de 1 y 100 generaciones no aportan evidencia cuantitativa útil). Este informe no dispone de los valores numéricos exactos de aptitud en cada generación, dado que estos no fueron proporcionados en formato tabular ni impreso, sino únicamente como gráficas, por lo que cualquier afirmación sobre convergencia se realiza a partir de la inspección visual de dichas gráficas, y se distingue explícitamente de una conclusión cuantitativa.

Etapa inicial (1 a 100 generaciones). La ausencia de cambio visual en la imagen reconstruida sugiere que, en un número tan reducido de generaciones y con un porcentaje de mutación tan bajo (0,01 %), la población no ha tenido oportunidad de alejarse de manera perceptible de su estado de inicialización aleatoria.

Etapa intermedia (1.000 a 10.000 generaciones). La gráfica de aptitud del experimento de 1.000 generaciones muestra un crecimiento sostenido, sin signos de estancamiento dentro del rango representado; la gráfica del experimento de 10.000 generaciones muestra igualmente un crecimiento sostenido a lo largo de todo su eje horizontal, con una curvatura que se aplana levemente hacia el final del rango representado, lo que constituye un indicio visual, no una demostración cuantitativa, de una posible desaceleración en la tasa de mejora de la aptitud a medida que el número de generaciones aumenta.

Etapa avanzada (25.000 a 50.000 generaciones). Las gráficas de aptitud de estos dos experimentos muestran un patrón similar al de la etapa intermedia: crecimiento sostenido con una curvatura que se aplana de manera visible hacia las últimas generaciones representadas en cada gráfica, sin llegar a mostrar una meseta completamente horizontal (estancamiento total). Esta observación visual es compatible con, pero no demuestra de forma cuantitativa, la hipótesis de que el algoritmo se encuentra en una fase de rendimientos decrecientes, en la que cada generación adicional aporta una mejora de aptitud proporcionalmente menor que en etapas anteriores.

Posible convergencia y posible estancamiento. Con los datos disponibles no es posible determinar si el algoritmo alcanzaría, en algún número finito adicional de generaciones, un estado de convergencia completa (aptitud máxima estable) o si, por el contrario, se encuentra en un régimen de mejora asintótica muy lenta debido al porcentaje de mutación extremadamente bajo configurado (0,01 %, equivalente a solo 3 genes mutados por individuo y por generación sobre un total de 30.000). Esta limitación de mutación podría, en principio, ralentizar de forma considerable la exploración de nuevas combinaciones de valores de gen no presentes ya en la población, favoreciendo una convergencia lenta y progresiva en lugar de una convergencia rápida seguida de estancamiento. Se trata, no obstante, de una interpretación razonada a partir del diseño del algoritmo, y no de una conclusión demostrada mediante el análisis estadístico de los valores de aptitud generación a generación, información que no está disponible en los archivos proporcionados.

Relación entre mejora visual y costo computacional. Al contrastar el análisis visual de la sección 13 con los tiempos de la sección 10, se observa que la transición más significativa en términos de calidad visual, la aparición de una silueta reconocible del objeto, ocurre entre los experimentos de 1.000 y 10.000 generaciones (es decir, se requiere multiplicar por 10 el número de generaciones, y el tiempo, de 120 a 1.020 segundos, un incremento de 8,5 veces, para obtener dicha mejora). En contraste, la transición de 25.000 a 50.000 generaciones, que multiplica igualmente el tiempo de ejecución (de 2.760 a 5.400 segundos, 1,96 veces) produce, de acuerdo con el análisis visual de la sección 13.5, una mejora perceptible pero considerablemente menor en magnitud que la observada en la transición anterior. Esta observación, estrictamente cualitativa y basada en inspección visual, es coherente con la noción de rendimientos decrecientes que se retoma en la discusión de la sección 16.

16. Discusión

¿El aumento de generaciones produce una mejora visible? Sí. El análisis visual desarrollado en la sección 13 muestra, de manera inequívoca, una progresión desde el ruido de color puro (1, 100 y 1.000 generaciones) hasta una aproximación reconocible de la forma y de algunas variaciones tonales de la fruta original (10.000, 25.000 y 50.000 generaciones). Esta mejora es consistente con el fundamento teórico expuesto en la sección 3: a medida que se ejecutan más ciclos de selección, cruce y mutación, los

individuos con menor error respecto de la imagen objetivo tienen mayor probabilidad de propagar sus genes a través de la población.

¿La mejora visual justifica el aumento del tiempo de ejecución? La respuesta depende del criterio de evaluación adoptado. Si el objetivo es obtener una silueta reconocible del objeto, el salto de 1.000 a 10.000 generaciones (multiplicando por 8,5 el tiempo de ejecución) resulta claramente justificado, dado que marca la transición entre el ruido total y la aparición de estructura reconocible. En cambio, si el objetivo es una reproducción de alta fidelidad, con colores y texturas fieles al original, ni siquiera el experimento de 50.000 generaciones logra dicho resultado (véase sección 13.5), lo que sugiere que, para ese objetivo más exigente, el tiempo adicional invertido en generaciones avanzadas produce mejoras cada vez menores en proporción al costo computacional adicional.

¿Qué ocurre con el rendimiento al pasar de 1.000 a 10.000 generaciones? Se observa la transición cualitativa más importante de todo el experimento: de una imagen sin estructura reconocible a una con silueta identificable. En términos de tiempo, el incremento es de 120 a 1.020 segundos (8,5 veces), frente a un incremento de 10 veces en el número de generaciones, lo que indica una eficiencia temporal razonable (el tiempo crece algo menos que proporcionalmente al número de generaciones) para una mejora visual de alto impacto relativo.

¿Qué ocurre al pasar de 10.000 a 50.000 generaciones? El tiempo de ejecución se multiplica por aproximadamente 5,3 veces (de 1.020 a 5.400 segundos), mientras que el número de generaciones se multiplica por 5 veces, una relación prácticamente proporcional. Sin embargo, la mejora visual observada en este tramo —el refinamiento del contorno y la aparición de variaciones tonales internas— es de una magnitud perceptualmente menor que la transición anterior (de 1.000 a 10.000), lo que constituye, desde el punto de vista cualitativo, un indicio de rendimientos decrecientes en la relación entre tiempo invertido y mejora visual obtenida.

¿Se observa una mejora proporcional al aumento del tiempo? No de manera uniforme a lo largo de todo el experimento. Mientras que el tiempo de ejecución crece de forma aproximadamente lineal con el número de generaciones (sección 10.2), la mejora visual percibida no sigue el mismo patrón: es prácticamente nula entre 1 y 1.000 generaciones, muy marcada entre 1.000 y 10.000, y de nuevo más moderada entre

10.000 y 50.000. Esta falta de proporcionalidad es consistente con el comportamiento típico de los procesos de optimización iterativa, en los que las primeras mejoras sustanciales suelen requerir superar una fase de «calentamiento» exploratorio, seguida de una fase de mejoras rápidas y, finalmente, una fase de ajuste fino con rendimientos decrecientes.

¿Existe evidencia visual de rendimientos decrecientes? Sí, en el sentido cualitativo señalado repetidamente en este informe: la diferencia visual entre las imágenes de 25.000 y 50.000 generaciones es notoriamente menor que la diferencia entre las imágenes de 10.000 y 25.000 generaciones, pese a que el incremento relativo de tiempo invertido en ambos tramos es de magnitud similar (2,71 y 1,96 veces, respectivamente). Se insiste en que esta es una observación visual y no una medición cuantitativa de una métrica objetiva de similitud (como el error cuadrático medio o el índice de similitud estructural SSIM), métricas que no fueron calculadas en el proyecto original ni pueden calcularse de forma retroactiva sin acceso a los valores numéricos de los píxeles reconstruidos en cada experimento con precisión suficiente.

¿Qué factores del algoritmo explican los resultados obtenidos? Principalmente tres factores, todos ellos documentados en las secciones 6 y 14: (1) el número de genes del cromosoma (30.000), que define un espacio de búsqueda de alta dimensionalidad; (2) el tamaño de población reducido (20 individuos), que limita la diversidad genética explorable en paralelo; y (3) el porcentaje de mutación extremadamente bajo (0,01 %, aproximadamente 3 genes por individuo y por generación), que ralentiza la introducción de nueva variabilidad genética y, en consecuencia, exige un número muy elevado de generaciones para lograr cambios visuales sustanciales.

¿Qué limitaciones presenta el experimento? Entre las principales limitaciones identificadas se encuentran: la ausencia de métricas objetivas de similitud (por ejemplo, SSIM o error cuadrático medio) que permitan cuantificar, más allá de la apreciación visual, el grado de aproximación logrado en cada experimento; la ausencia de valores numéricos impresos de aptitud (solución encontrada, generación de convergencia) en los archivos entregados; la falta de información sobre el hardware empleado, lo que impide generalizar los tiempos de ejecución a otros entornos; y la inconsistencia detectada en las gráficas de aptitud de los experimentos de 1 y 100 generaciones, que

reduce la evidencia cuantitativa disponible para las primeras etapas del proceso evolutivo.

¿Qué mejoras podrían realizarse en futuros experimentos? Las recomendaciones específicas se desarrollan en detalle en la sección 18; entre ellas destacan el cálculo sistemático de métricas objetivas de similitud, la instrumentación explícita del tiempo de ejecución, la exploración de distintos tamaños de población y porcentajes de mutación, y la evaluación de mecanismos de paralelización para reducir el costo computacional de los experimentos con un número elevado de generaciones.

17. Conclusiones

A partir del análisis técnico del código fuente, de los seis experimentos documentados en el archivo tiempos.txt y de las doce imágenes de resultados proporcionadas, se formulan las siguientes conclusiones, todas ellas sustentadas exclusivamente en la evidencia disponible en los archivos del proyecto.

Sobre el funcionamiento del algoritmo. El algoritmo genético implementado mediante PyGAD, en conjunto con el módulo GARI para la conversión imagen–cromosoma, funciona conforme a lo previsto por su diseño teórico: una función de aptitud basada en la suma de diferencias absolutas respecto de la imagen objetivo, transformada a un criterio de maximización, permite guiar la evolución de una población de 20 individuos de 30.000 genes cada uno hacia soluciones progresivamente más parecidas a la imagen original.

Sobre la evolución de la imagen. La evidencia visual reunida en las secciones 12 y 13 muestra una progresión clara, aunque no lineal, desde el ruido de color puro (experimentos de 1, 100 y 1.000 generaciones) hasta una aproximación reconocible, aunque todavía imperfecta en color y detalle, de la forma general de la fruta original (experimentos de 10.000, 25.000 y 50.000 generaciones). En ningún experimento se logró una reproducción visualmente indistinguible de la imagen original.

Sobre la influencia del número de generaciones. El número de generaciones resultó ser el factor más determinante, entre los analizados, tanto para la calidad visual del resultado como para el tiempo total de ejecución. La transición cualitativa más relevante, la aparición de una silueta reconocible, se produjo entre los experimentos de 1.000 y 10.000 generaciones, mientras que los incrementos posteriores (hasta 50.000 generaciones) produjeron mejoras visuales de menor magnitud relativa, un patrón compatible, desde el punto de vista cualitativo, con rendimientos decrecientes.

Sobre el tiempo de ejecución. El tiempo de ejecución creció de 20 segundos (1 generación) a 5.400 segundos (50.000 generaciones), es decir, un factor de 270 veces, frente a un factor de 50.000 veces en el número de generaciones. Este comportamiento se explica por la existencia de una sobrecarga fija de inicialización, común a todos los experimentos, cuyo peso relativo disminuye a medida que aumenta el número de

generaciones, y por un costo por generación que se estabiliza, a partir de 1.000 generaciones, en un rango de entre 0,102 y 0,120 segundos.

Sobre la complejidad computacional. La complejidad del problema abordado está determinada, principalmente, por el número de genes del cromosoma (30.000, resultado de 100 × 100 píxeles × 3 canales de color), el tamaño de la población (20 individuos) y el número de generaciones ejecutadas. El costo de cada evaluación de aptitud crece linealmente con el número de genes, por lo que el uso de imágenes de mayor resolución incrementaría de forma proporcional y, en el caso de imágenes cuadradas, cuadráticamente respecto del lado de la imagen el costo computacional total del experimento.

Sobre la utilidad de los algoritmos genéticos para la reproducción de imágenes. El experimento analizado demuestra, de manera práctica y pedagógica, que un algoritmo genético relativamente simple, sin operadores especializados de cruce o mutación adaptados específicamente al dominio de imágenes, es capaz de aproximar progresivamente una imagen objetivo a partir de una población inicial completamente aleatoria, evidenciando de forma tangible los conceptos de selección, cruce y mutación descritos en el marco teórico. No obstante, la eficiencia de este enfoque, en su configuración actual, es limitada frente a métodos de reconstrucción de imágenes basados en optimización por gradiente o en representaciones más compactas, dado el elevado número de generaciones y el tiempo de cómputo requeridos para obtener incluso una aproximación parcial.

Sobre las limitaciones encontradas. El análisis estuvo limitado por: la ausencia de métricas objetivas de similitud entre las imágenes reconstruidas y la imagen original; la ausencia de valores numéricos impresos de aptitud; la falta de información sobre el hardware de ejecución; y las inconsistencias detectadas en el nombre y la procedencia real de la imagen original, así como en las gráficas de aptitud de los experimentos de 1 y 100 generaciones. Estas limitaciones fueron documentadas explícitamente a lo largo del informe, en cumplimiento del principio de no atribuir al proyecto datos, parámetros o resultados no verificables.

18. Recomendaciones

A partir de las limitaciones y hallazgos descritos en las secciones anteriores, se formulan las siguientes recomendaciones técnicas para la realización de experimentos futuros que profundicen o mejoren el proyecto analizado.

Tabla 8. Recomendaciones técnicas para experimentos futuros

Recomendación	Descripción
Optimización del código	Instrumentar el tiempo de ejecución de manera explícita (por ejemplo, con time.perf_counter()) y evaluar el uso del parámetro fitness_batch_size de PyGAD para vectorizar la evaluación de la población, reduciendo la sobrecarga de invocaciones individuales de la función de aptitud.
Uso de imágenes de diferentes resoluciones	Repetir el experimento con imágenes de menor (por ejemplo, 50 × 50) y mayor (por ejemplo, 200 × 200) resolución, con el fin de cuantificar empíricamente la relación entre el número de genes y el tiempo de ejecución por generación, contrastando dicha relación con el análisis teórico presentado en la sección 14.
Variación del tamaño de población	Explorar valores de sol_per_pop mayores (por ejemplo, 50 o 100 individuos) y evaluar si un número menor de generaciones, combinado con una población más diversa, permite alcanzar un nivel de aproximación visual similar al obtenido en este proyecto con 50.000 generaciones y una población de 20 individuos, en un tiempo total de ejecución comparable o menor.
Variación de los parámetros de mutación	Evaluar sistemáticamente distintos valores de mutation_percent_genes (por ejemplo, 0,1 %; 1 %; 5 %) y de mutation_type, dado que el valor extremadamente bajo utilizado en este proyecto (0,01 %) es, según el análisis de la sección 14, uno de los factores que más contribuyen a la lentitud de la convergencia visual observada.
Evaluación con diferentes funciones de aptitud	Comparar la función de aptitud basada en la suma de diferencias absolutas (empleada en este proyecto) con otras métricas de error, como el error cuadrático medio (MSE) o medidas perceptuales de

	similitud estructural (SSIM), que podrían guiar la evolución hacia soluciones perceptualmente más fieles a la imagen objetivo.
Uso de métricas objetivas de similitud	Incorporar, en cada experimento, el cálculo de métricas cuantitativas de similitud entre la imagen reconstruida y la imagen original (por ejemplo, PSNR, SSIM o el propio valor de aptitud impreso numéricamente), de manera que el análisis de convergencia pueda sustentarse en datos cuantitativos y no únicamente en la inspección visual, como fue necesario en el presente informe.
Paralelización	Evaluar el uso de los mecanismos de procesamiento paralelo ofrecidos por PyGAD (parámetro parallel_processing) para distribuir la evaluación de la población entre varios núcleos de procesamiento, con el objetivo de reducir el tiempo total de ejecución en experimentos de un número elevado de generaciones.
Comparación con otros algoritmos de optimización	Contrastar el desempeño del algoritmo genético frente a otras técnicas de optimización metaheurística (por ejemplo, recocido simulado, optimización por enjambre de partículas o estrategias evolutivas) o frente a métodos de reconstrucción de imágenes basados en descenso de gradiente, con el fin de situar los resultados obtenidos en un marco comparativo más amplio.
Registro y documentación reproducible	Mantener un registro estructurado (por ejemplo, en formato de hoja de cálculo o archivo CSV) de los parámetros exactos, los tiempos medidos automáticamente y los valores numéricos de aptitud de cada ejecución, evitando las inconsistencias de nomenclatura y de contenido detectadas en este informe entre los archivos de imagen y el archivo de tiempos.

19. Referencias bibliográficas

[1] J. H. Holland, Adaptation in Natural and Artificial Systems. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[2] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Reading, MA, USA: Addison-Wesley, 1989.

[3] M. Mitchell, An Introduction to Genetic Algorithms. Cambridge, MA, USA: MIT Press, 1998.

[4] M. Paulinas and A. Ušinskas, "A survey of genetic algorithms applications for image enhancement and segmentation," Inf. Technol. Control, vol. 36, no. 3, pp. 278–284, 2007.

[5] S. Mirjalili, J. Song Dong, A. S. Sadiq, and H. Faris, "Genetic algorithm: Theory, literature review, and application in image reconstruction," in Nature-Inspired Optimizers, Studies in Computational Intelligence, vol. 811, S. Mirjalili, J. Song Dong, and A. Lewis, Eds. Cham, Switzerland: Springer, 2020, pp. 69–85, doi: 10.1007/978-3-030-12127-3_5.

[6] A. F. Gad, "PyGAD: An intuitive genetic algorithm Python library," arXiv:2106.06158 [cs.NE], Jun. 2021.

[7] PyGAD, "pygad.GA class — PyGAD 3.x documentation," pygad.readthedocs.io. [Online]. Available: https://pygad.readthedocs.io/en/stable/pygad.html

[8] PyGAD, "Reproducing images — PyGAD 3.x documentation," pygad.readthedocs.io. [Online]. Available: https://pygad.readthedocs.io/en/stable/pygad_example_reproducing_images.html

[9] A. F. Gad, "GARI: Genetic Algorithm for Reproducing Images," GitHub repository. [Online]. Available: https://github.com/ahmedfgad/GARI

[10] M. Oltean, "Fruits-360 dataset," Mendeley Data, V5, 2025. [Online]. Available: https://data.mendeley.com/datasets/rp73yg93n8/5

[11] Fruits-360, "fruits-360-100x100," GitHub repository. [Online]. Available: https://github.com/fruits-360/fruits-360-100x100

20. Anexos

Se incluyen a continuación los fragmentos de código más relevantes del proyecto, ya presentados y explicados en la sección 7, reunidos aquí a modo de referencia rápida, junto con la tabla completa de tiempos y una tabla resumen de todas las imágenes entregadas.

20.1. Anexo A. Función de aptitud completa (fitness_fun)

```
def fitness_fun(ga_instance, solution, solution_idx):
    fitness = numpy.sum(numpy.abs(target_chromosome-solution))

    # Negating the fitness value to make it increasing rather than decreasing.
    fitness = numpy.sum(target_chromosome) - fitness
    return fitness
```

20.2. Anexo B. Configuración completa de pygad.GA

```
ga_instance = pygad.GA(num_generations=50000,
                       num_parents_mating=10,
                       fitness_func=fitness_fun,
                       sol_per_pop=20,
                       num_genes=target_im.size,
                       init_range_low=0.0,
                       init_range_high=1.0,
                       mutation_percent_genes=0.01,
                       mutation_type="random",
                       mutation_by_replacement=True,
                       random_mutation_min_val=0.0,
                       random_mutation_max_val=1.0)
ga_instance.run()
```

20.3. Anexo C. Funciones principales de gari.py

```
import numpy
import functools
import operator

def img2chromosome(img_arr):
    """
    Represents the image as a 1D vector (chromosome).
    img_arr: Input image (2D or 3D) as a NumPy array.
    Returns a 1D vector representing the image (chromosome).
    """
    return numpy.reshape(a=img_arr, newshape=(functools.reduce(operator.mul,
img_arr.shape)))

def chromosome2img(vector, shape):
    """
    Converts a 1D vector (chromosome) into a 2D or 3D image.
    vector: 1D vector to be converted into an image.
    shape: Target shape of the image.
    Returns the reconstructed image as a NumPy array.
    """
    if len(vector) != functools.reduce(operator.mul, shape):
        raise ValueError(f"A vector of length {len(vector)} into an array of shape
{shape}.")
    return numpy.reshape(a=vector, newshape=shape)
```

20.4. Anexo D. Tabla completa de tiempos de ejecución (archivo tiempos.txt)

Tabla 9. Tabla completa de tiempos de ejecución (ortografía corregida; valores originales conservados)

Generaciones	Tiempo (formato original, corregido)	Tiempo en segundos
1	20 segundos	20 s
100	21 segundos	21 s

1.000	2 minutos	120 s
10.000	17 minutos	1.020 s
25.000	46 minutos	2.760 s
50.000	1 hora con 30 minutos	5.400 s

20.5. Anexo E. Relación de archivos de imagen entregados

Tabla 10. Relación completa de archivos de imagen entregados en el proyecto

Archivo	Tipo de contenido	Experimento asociado	Observación
1.png	Gráfica de aptitud	Experimento 1 generación	Eje horizontal de 0 a 1 (véase inconsistencia, sección 9)
11.png	Imagen reconstruida	Experimento 1 generación	Ruido de color puro
100.png	Gráfica de aptitud	Experimento 100 generaciones	Eje horizontal de 0 a 1 (inconsistencia, sección 9)
101.png	Imagen reconstruida	Experimento 100 generaciones	Ruido de color puro
1000.png	Gráfica de aptitud	Experimento 1.000 generaciones	Eje horizontal de 0 a 1.000, coherente
1001.png	Imagen reconstruida	Experimento 1.000 generaciones	Predomina el ruido; sin estructura clara
10000.png	Gráfica de aptitud	Experimento 10.000 generaciones	Eje horizontal de 0 a 10.000, coherente
10001.png	Imagen reconstruida	Experimento 10.000 generaciones	Emerge silueta redondeada del objeto
25000.png	Gráfica de aptitud	Experimento 25.000 generaciones	Eje horizontal de 0 a 25.000, coherente

25001.png	Imagen reconstruida	Experimento 25.000 generaciones	Contorno más definido; variación tonal interna
50000.png	Gráfica de aptitud	Experimento 50.000 generaciones	Eje horizontal de 0 a 50.000, coherente
50001.png	Imagen reconstruida	Experimento 50.000 generaciones	Mejor aproximación visual obtenida en el proyecto
fruit.jpg	Imagen original (target_im)	Referencia para todos los experimentos	100 × 100 píxeles, RGB, 24 bits